

COURSE SUMMARY

HELM

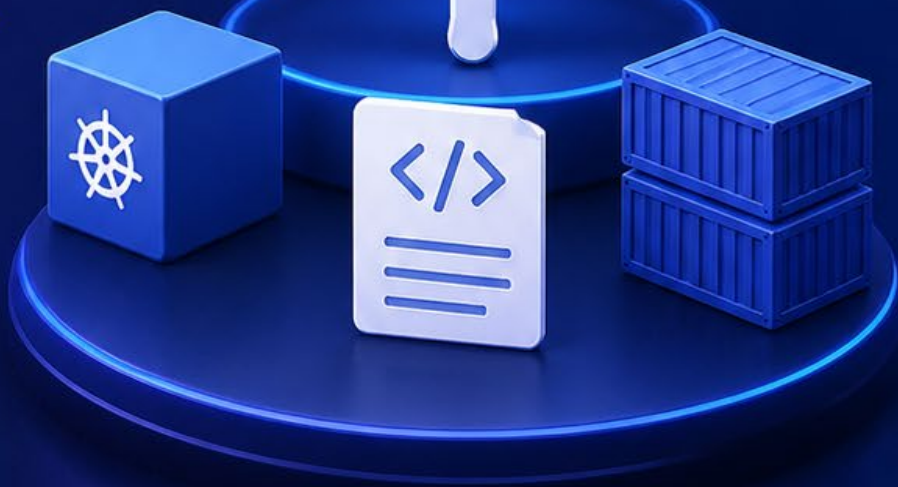
PACKAGE MANAGER FOR KUBERNETES



Comprehensive Summary of
Concepts, Best Practices,
and Key Takeaways for the
Helm Course



Youssef ElZarka



Instructor
Mumshad Mannambeth



PACKAGE



UPGRADE



MANAGE

PACKAGE • DEPLOY • UPGRADE • ROLLBACK
SIMPLIFY KUBERNETES APPLICATION MANAGEMENT

Contents

Introduction.....	2
What is Helm?.....	4
Installation & Configuration	7
Helm2 vs Helm3.....	9
Helm Components	12
Helm Charts	17
Working with Helm (Basics)	22
Customizing Chart Parameters	25
Lifecycle Management with Helm	27
Writing Helm Charts.....	31
Making sure Chart is working as intended.....	34
Function	37
Pipelines with Functions	39
Conditionals	41
With Blocks	44
Loops & Ranges.....	47
Named Templates	49
Chart Hooks.....	52
Packaging and Signing Charts	55
Uploading Charts	58

- ❑ Helm هو أداة لإدارة التطبيقات على Kubernetes
- ❑ الفكرة الأساسية منه إنه يسهل عملية نشر التطبيقات وإدارة دورة حياتها، بمعنى أنك بدل ما تكتب كل الموارد زي (Pods, Services, ConfigMaps,) يدويا ، فتقدر تستخدم Charts جاهزة أو تطور Charts خاصة بك لتبسيط النشر والتحديث والصيانة.
- ❑ الهدف الأساسي من الكورس ده هو تعليمك:
 - أساسيات Helm
 - طريقة التثبيت والبدء في استخدامه.
 - التعرف على المكونات المختلفة مثل Charts, Repos, Releases, Revisions
 - تطوير Charts من الصفر واستخدام الوظائف الشرطية، الحلقات، ال Hooks ، وغيرها.

الشرح خطوة خطوة

1 التعريف بالموضوع الأساسي للكورس

- Helm كأداة لتسهيل النشر وإدارة التطبيقات على Kubernetes
- التركيز على تقليل التعقيد عند التعامل مع التطبيقات متعددة الموارد.

2 تغطية محتوى الكورس الأساسي

- تعلم أساسيات Helm
- التثبيت والبدء في الاستخدام.
- التعرف على المكونات:
 - Charts : حزم التطبيقات التي تحتوي على الموارد المطلوبة.
 - Repos : مستودعات لتخزين ومشاركة Charts
 - Releases : نسخ نشطة من تطبيقاتك عند نشرها.
 - Revisions : إصدارات سابقة من النشر يمكن الرجوع إليها.

3 تطوير Charts

- تعلم كيفية إنشاء Charts من الصفر.
- استخدام الوظائف:
 - Functions : لتسهيل العمليات داخل القوالب.
 - Pipelines : تمرير البيانات بين الوظائف.
 - Conditionals : تنفيذ الأكواد وفق شروط معينة.
 - With blocks : لتبسيط الوصول للبيانات المتداخلة.
 - Ranges : لتكرار العناصر في القوالب.
 - Hooks : تنفيذ أوامر قبل أو بعد نشر التطبيق.

4 تعبئة وتوقيع Charts

- تجهيز Chart للنشر في المستودعات.
- توقيع Chart لضمان سلامة البيانات والمصادقية.

• Best Practices

- حافظ على تنظيم Charts بشكل واضح ومقسّم لمجلدات templates, values, charts
- استخدم versioning دائم لكل Chart لتسهيل الرجوع أو التحديث.
- دائماً اختبر Charts في بيئة تطوير قبل نشرها على الإنتاج.

• Real Use Cases

- نشر تطبيقات متكاملة مثل WordPress + MySQL باستخدام Chart واحد.
- إدارة تحديثات مستمرة للتطبيقات بدون توقف الخدمة.
- تخزين Charts في مستودعات داخلية للشركة لتوحيد النشر بين التيمات.

• Tricks

- يمكن استخدام helm template لمعاينة الموارد الناتجة قبل نشرها.
- استخدم --dry-run للتحقق من التغييرات قبل تنفيذها على الكلاستر.
- استغل Hooks لتشغيل أوامر قبل/بعد النشر مثل التهيئة أو تنظيف الموارد المؤقتة.

● الملخص النهائي

❖ Helm هو أداة قوية لتبسيط إدارة التطبيقات على Kubernetes

❖ من خلال هذا الكورس، ستتعلم:

1. المفاهيم الأساسية والمكونات Charts, Repos, Releases, Revisions
2. كيفية إنشاء Charts من الصفر واستخدام الوظائف الشرطية والحلقات و Hooks
3. تعبئة، توقيع، ونشر Charts بأمان.

What is Helm?

● مقدمة بسيطة: ليه Helm أصلاً موجود؟

- ❑ Kubernetes رهيب في إدارة البنية التحتية المعقدة 🤖 .. بس الموضوع ساعات بيبقى معقد علينا لما التطبيق بيقي كبير
- ❑ يعني لو إحنا بننشر أي تطبيق على الكلاستر، فده عادة بيقي مجموعة Objects مترابطة:
 - Pods : اللي هي السيرفرات اللي هتشغل التطبيق بتاعك.
 - Persistent Volumes : تخزين البيانات بشكل دائم.
 - Persistent Volume Claims (PVCs) : زي الطلب على التخزين.
 - Services : عشان نخلي التطبيق يوصل للننت أو جوه الكلاستر.
 - Secrets : لتخزين كلمات السر أو معلومات حساسة.
 - Jobs or CronJobs : لو محتاجين حاجات دورية زي النسخ الاحتياطي.
- ❑ لو حاولت تعمل ده كله يدوي، هتحتاج تعمل كل Resource في YAML منفصل + تعمل **kubectl apply** لكل واحد وطبعاً ده تعب جامد، خصوصاً لو التطبيق كبير.



◆ المشكلة من غير Helm

- لو التطبيق فيه 20 or 30 YAMLFILE ← التعديل بقي كابوس. 🤖
- لو عايز تزود التخزين أو تعدل Password أو أي حاجة ثانية ← هتفتح كل ملف YAML وتعدله.
- بعد شهرين لو عايز تحدث حاجة أو تسمح التطبيق ← هتضطر تفكر كل Object موجود فين وتعمله delete يدوي.
- حتى لو حبيت تحط كل حاجة في YAML واحد ← فالملف هابقي طويل جداً وصعب تلاقي الحاجة اللي محتاج تعدلها.

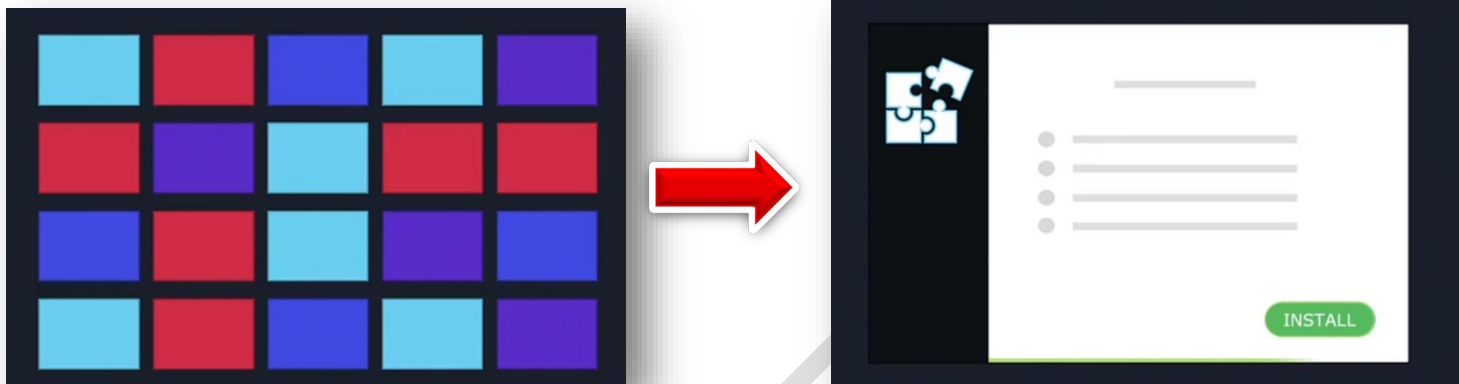
✚ باختصار : Kubernetes بيهتم بكل Object لوحده، ومش بيشوفهم كاتطبيق كامل.

• Kubernetes Helm = Package Manager

- يعني ببشوف التطبيق كـ Package كامل مش مجموعة Objects منفصلة.
- تقدر تعمل Install للتطبيق كله بأمر واحد، مهما كان فيه Resources كتير.
- نفس الكلام لـ Upgrade أو Rollback أو Uninstall

❖ مثال عملي:

- تخيل لعبة كمبيوتر فيها آلاف الملفات (كود، صور، أصوات، إعدادات...) بدل ما تحمل كل ملف لوحده، فيه Installer بيعمل كل ده بأمر واحد.
- Helm بقي بيعمل نفس الحاجة مع Kubernetes manifests



◆ إزاي Helm بيحل المشاكل؟

1. تثبيت التطبيق بالكامل
○ **Helm install** ← Helm يعرف كل Object محتاج بنضاف للكلاستر.
2. تخصيص الإعدادات بسهولة

○ بدل ما تعدل 10 YAML ، عندك ملف واحد values.yaml تحط فيه كل التغييرات:

- حجم الـ Persistent Volumes
- اسم الموقع
- كلمات سر الداتا بيز
- أي إعدادات تانية

3. تحديث التطبيق Upgrade

- **Helm upgrade** ← Helm هيحدد أي Objects محتاجة تعديل تلقائيًا.

4. إدارة الإصدارات Revisions

- **helm rollback** ← Helm بيخزن كل التغييرات، ممكن ترجع لأي نسخة سابقة بسهولة.

5. إلغاء التثبيت Uninstall

- **Helm uninstall** ← Helm يعرف كل Resources اللي جزء من التطبيق ويحذفهم كلها بأمر واحد.

◆ إضافات مهمة

- Helm Charts : دي حزم التطبيقات اللي جواها كل الموارد المطلوبة. ممكن تاخد Chart جاهز أو تعمل Chart خاص بيك.
- Repositories : مستودعات Charts ، زي ما GitHub للتطبيقات، تقدر تخزن Charts عامة أو خاصة بالشركة.
- Release : النسخة اللي انت منزلها من التطبيق.
- Hooks : أوامر تشتغل قبل أو بعد تثبيت/تحديث التطبيق، زي Job للنسخ الاحتياطي قبل التحديث.
- Template Functions : أدوات لتسهيل تخصيص القيم جوه Charts ، مش بس كتابة قيم ثابتة.

```
values.yaml
wordpressUsername: user
## Application password
## Defaults to a random 10-character alphanumeric string if not set
## ref: https://github.com/bitnami/bitnami-
##
# wordpressPassword:
## Admin email
## ref: https://github.com/bitnami/bitnami-
##
wordpressEmail: user@example.com
## First name
## ref: https://github.com/bitnami/bitnami-
##
wordpressFirstName: FirstName
## Last name
## ref: https://github.com/bitnami/bitnami-
##
wordpressLastName: LastName
## Blog name
## ref: https://github.com/bitnami/bitnami-
##
wordpressBlogName: User's Blog
WORDPRESS
```

• : Best Practices

- خلي كل إعدادات التخصيص في values.yaml
- استخدم أسماء واضحة للـ Releases : dev-wordpress, prod-wordpress
- جرب Charts على Dev Cluster قبل الإنتاج.

• : Real Use Cases

- نشر WordPress + MySQL بأمر واحد بدل 20 YAML مختلف.
- تحديث تطبيقات في بيئات مختلفة بدون أي downtime
- مشاركة Charts داخل الشركة لتسهيل النشر للفرق المختلفة.

• : Tricks

- helm template لمعاينة Resources قبل التثبيت.
- --dry-run للتأكد من أي تغييرات قبل التطبيق.
- استخدم Hooks لتشغيل Jobs أو إعداد ConfigMaps قبل أو بعد التثبيت.

● الملخص النهائي

Helm بيخليك:

- تشوف التطبيق كـ Package كامل مش مجموعة Objects منفصلة.
 - تثبت، تحدث، أو تحذف بأمر واحد.
 - تعدل إعداداتك في مكان واحد (values.yaml)
 - تحتفظ بسجل التغييرات للرجوع لأي إصدار سابق.
 - تقلل الحمل على المهندسين، وتجنب الأخطاء في التطبيقات الكبيرة.
- 🌟 الخلاصة : Helm يوفر وقتك، يقلل الأخطاء، ويخلي إدارة التطبيقات على Kubernetes أسهل بكثير.

Installation & Configuration

● مقدمة عامة

- ❑ قبل ما نثبت Helm ، في شوية حاجات لازم نتأكد منها:
 1. عندك Kubernetes Cluster شغال.
 2. متثبت kubectl على جهازك ومضبوط بحيث يقدر يتصل بالكلاستر.
 3. ملف kubeconfig مضبوط، ده اللي فيه بيانات الدخول للكلاستر والـ context اللي هتشتغل عليه.
- ❑ Helm بيشتغل عادي على Linux ، Windows ، Mac OS ، لكن هنا هنتكلم عن Linux

◆ الشرح خطوة بخطوة

1 تثبيت Helm على Linux باستخدام Snap

- ❖ لو جهازك فيه Snap ، تقدر تثبت Helm كده : **sudo snap install helm --classic**
- **--classic** ← بيدي Helm صلاحيات أكبر للوصول للـ kubeconfig في الـ home directory
- بدون الـ --classic ، Helm ممكن يلاقي صعوبة في الاتصال بالكلاستر.

2 تثبيت Helm على أنظمة Debian / Ubuntu باستخدام APT

1. أضف الـ sources list & key :
 - **curl https://baltocdn.com/helm/signing.asc | sudo apt-key add -**
 - **sudo apt-add-repository "deb https://baltocdn.com/helm/stable/debian/ all main"**
 - **sudo apt update**
 2. ثبت Helm : **sudo apt install helm**
- ✓ ملاحظات :
- تأكد دايماً من آخر نسخة متوافقة مع نظامك.
 - لو النسخة قديمة من Ubuntu/Debian ، ممكن تواجه مشاكل.

3 تثبيت Helm باستخدام PKG

- لو النظام بتاعك بيدعم PKG (زي MacOS أو بعض توزيعات Linux) : **sudo pkg install helm**
- دايماً راجع Documentation الرسمية حسب الـ OS والنسخة.

4 التحقق من التثبيت

- بعد ما تثبت Helm ، جرب : **helm version**
- هيرجع لك نسخة Helm المثبتة، وده بياكد أن كل حاجة مضبوطة وأن Helm يقدر يتصل بالكلاستر.

5 نصائح عملية قبل التثبيت

- تأكد من صلاحياتك (root or sudo)
- اتأكد إن kubectl متصل بالكلاستر: **kubectl get nodes**
- لو رجعتك Nodes ← تمام ... لو لا ← ظبط kubeconfig
- لو على بيئة Production ، اعمل backup للـ kubeconfig وأي ملفات مهمة.

Best Practices

- استخدم **--classic** مع Snap عشان Helm يوصل للملف kubeconfig
- دايماً راجع Documentation الرسمية قبل التثبيت على أي OS
- جرب Helm على بيئة Dev قبل الإنتاج.

Real Use Cases

- DevOps engineers بيستخدموا Helm على سيرفرات Ubuntu/Debian لأنه سهل وسريع.
- استخدام Snap مع **--classic** شائع على الأجهزة الشخصية أو VMs

Tricks

- لو عندك أكثر من Kubernetes cluster ، اتأكد إن Helm شغال على الـ context الصح.
- ممكن تعمل alias لـ kubectl و Helm لتسريع الأوامر.
- Helm بيخزن config داخلياً، فحافظ على تحديث الملفات لتجنب مشاكل الاتصال.

المخلص النهائي

1. Helm محتاج Kubernetes Cluster شغال + kubectl مضبوط + kubeconfig مضبوط.
 2. طرق التثبيت على Linux ← Snap ، APT ، PKG
 3. Snap أسهل + استخدم **--classic**
 4. تحقق من الاتصال بالكلاستر بعد التثبيت باستخدام **helm version** و **kubectl get nodes**
 5. بعد التثبيت، Helm جاهز لإدارة التطبيقات : تثبيت، تحديث، rollback ، وإلغاء التثبيت بسهولة.
- 💡 الخلاصة العملية: خطوة تثبيت Helm سهلة بس لازم كل حاجة قبلها مضبوطة عشان باقي الأوامر تمشي من غير مشاكل.

Helm2 vs Helm3

مقدمة عامة

- الجزء ده بيتكلم عن تطور Helm من النسخة 2 إلى النسخة 3، مع التركيز على:
 - تاريخ Helm والإصدارات المختلفة
 - المكونات الأساسية في كل نسخة، خصوصاً Tiller في Helm 2
 - آلية التحديث والـ Rollbacks للـ Charts
 - ميزة Three-Way Strategic Merge Patch في Helm 3
 - تحسينات الأمان بفضل RBAC
- الهدف : نفهم إزاي Helm 3 أصبح أسهل، أمان، وأكثر ذكاءً في إدارة Kubernetes

1 تاريخ Helm وتطوره

النسخة	تاريخ الإصدار
Helm 1.0	فبراير 2016
Helm 2.0	نوفمبر 2016
Helm 3.0	نوفمبر 2019

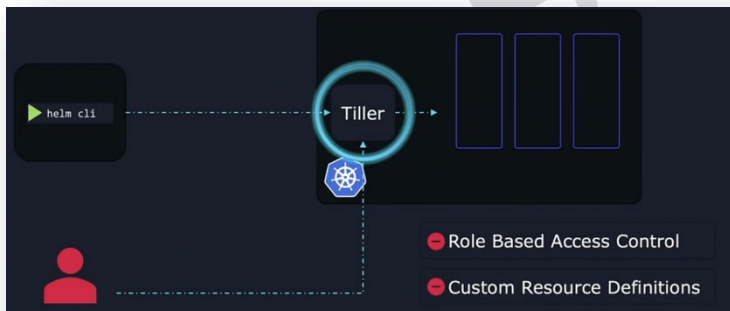
المهم هنا إن Helm كبر ونضج مع الوقت، ومع تطور Kubernetes نفسه، بقى فيه مميزات جديدة زي:

- RBAC (Role-Based Access Control)
 - CRDs (Custom Resource Definitions)
- المميزات دي سمحت لـ Helm 3 إنه يستغنى عن مكونات قديمة زي Tiller ويشغل بطريقة أنصف وأمان.

2 Helm 2 : مكونات النظام (Helm CLI Client + Tiller)

الفكرة الأساسية : في Helm 2 ، العمل كان مبني على مكونين:

- Helm Client (على جهازك المحلي) .. وده اللي بتكتب منه أوامر تثبيت أو ترقية Charts.
- Tiller (داخل Kubernetes Cluster) .. وده كان الكائن اللي بيتولى تنفيذ أوامر Helm على الكلاستر.



طريقة العمل في Helm 2 :

1. أنت تكتب أمر في الجهاز : **helm install "something"**

2. Helm CLI يرسل الطلب لـ Tiller

3. Tiller يتواصل مع Kubernetes ويعمل اللي انت طلبته

بالتالي، Tiller كان الوسيط الأساسي بينك وبين Kubernetes.

مشاكل Tiller :

- زيادة التعقيد ← وجود وسيط في النص بيزود خطوات ورسك.
- ضعف الأمان ← Tiller كان بيشتغل في God Mode، يعني:
 - يقدر يعمل أي حاجة في الكلاستر
 - أي مستخدم عنده وصول لـ Tiller يقدر يخرب كل حاجة بدون قيود .. وده كان كابوس أمني.

3 Helm 3 : التخلص من Tiller وتحسين الأمان

- ◀ شال Tiller للأبد.
- ◀ النتيجة:
- Helm CLI يبقى يتواصل مباشرة مع Kubernetes
- يبقى فيه اعتماد كامل على RBAC
- ◀ RBAC : 🔒
- RBAC يحدد مين مسموح له يعمل إيه داخل Kubernetes
- Helm 3 يبقى يمشي بنفس صلاحيات المستخدم
- سواء استخدمت helm أو kubectl ← الصلاحيات بتتطبق بنفس الطريقة
- 💡 ده خلى الأمان أفضل بكثير جداً.

4 Helm Helm Revisions وال Rollbacks

Helm بيوثق أي عملية بتحصل على ال release على شكل Revision :

- Install → Revision 1
 - Upgrade → Revision 2
 - Rollback → Revision 3
- كل Revision عبارة عن Snapshot لحالة ال release وقتها.

vs الفرق بين Helm 2 و Helm 3 في ال Rollbacks

❖ Helm 2 كان بيقارن بين :

- ال Chart الحالي
- وال Chart القديم

بس ماكاش بيبص على الحالة الحية في Kubernetes ..
وده يتسبب في مشاكل ضخمة

🔴 مثال : لو غيرت ال Deployment Image يدويًا بـ kubectl

- Helm 2 مش هيشوف التغيير ده
- rollback مش هيرجع الصورة زي زمان

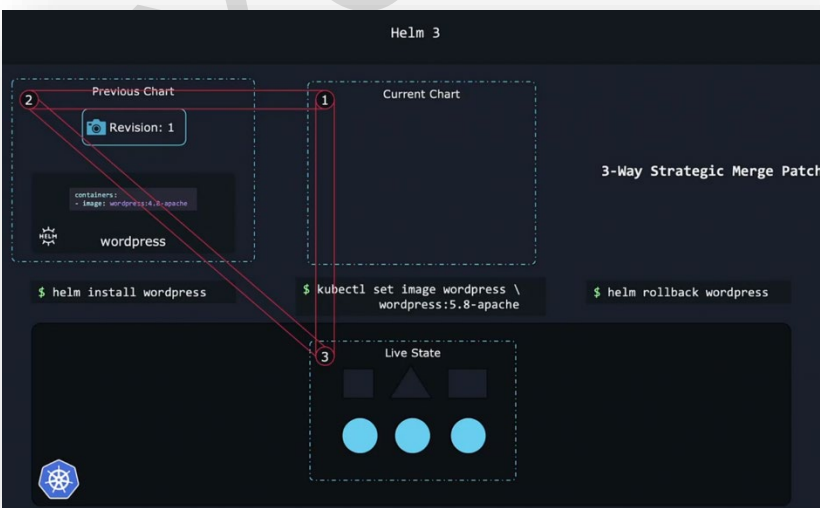
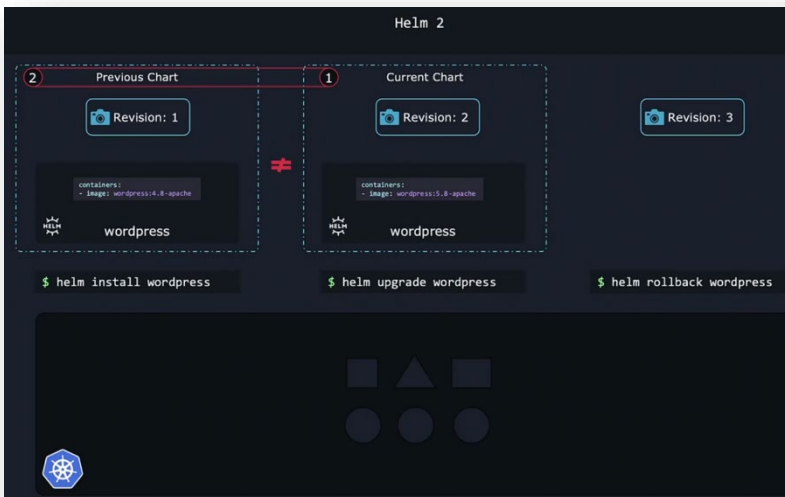
❖ لكن Helm 3 بيعمل مقارنة ثلاثية:

1. ال Chart الحالي
2. ال Chart اللي عايزين نرجع له
3. الحالة الحية Live State في Kubernetes

🌟 وده اللي اسمه : Three-Way Strategic Merge Patch

◀ يعني Helm 3 بيقدر يشوف:

- إيه تغيّر جوه ال Charts
 - وإيه تغيّر يدويًا جوه الكلاستر
 - ويقرر إزاي يرجع الأمور بأمان
- 🌟 وبالتالي ده بيخلي rollback في Helm 3 دقيق ومضمون.



: Helm 2

- كان بيمسح أي تعديلات يدوية حصلت على الكلاستر أثناء upgrade

: Helm 3

- يمشوف التعديلات اليدوية من خلال Live State
- وبالتالي يحافظ عليها أثناء الترقية

🔗 النتيجة:

مفیش فقد بيانات + مفیش surprises + المستخدم يتحكم في كل التفاصيل

6 Best Practices / Real Use Cases / Tricks

💡 Best Practices

- استخدم Helm 3 لأي مشروع جديد
- استخدم helm history قبل أي upgrade
- اربط صلاحيات مستخدمين Helm بـ RBAC
- احفظ values files في Git لسهولة التتبع

📁 Real Use Case

- تثبيت WordPress أو تطبيق معقد باستخدام Helm Chart
 - upgrade image version → revision 2
 - حد غير حاجة يدوياً Helm 3 → يلتقط التغيير
 - rollback ← يرجع بدقة بدون فقد التعديلات

🔧 Tricks

- **helm history <release>** ← لمراجعة كل الـ Revisions
- **helm rollback <release> <revision>** ← للرجوع لأي نسخة سابقة
- **helm diff upgrade** (لو سطبت helm-diff plugin) ← تشوف الفرق قبل التطبيق
- **helm get manifest <release>** ← تشوف ملفات الـ YAML اللي Helm ولدها
- استخدم قيم مختلفة لكل بيئة:
 - values-dev.yaml
 - values-stage.yaml
 - values-prod.yaml

📄 ملخص شامل

1. Helm 3 أبسط وأمن من Helm 2 لأنه حذف الـ Tiller وبقي يشتغل مباشرة مع Kubernetes
2. RBAC يحدد صلاحيات المستخدمين بدقة
3. Revisions و Rollbacks أنكى ← يقارن بين Chart القديم والجديد والحالة الحية
4. أي تغييرات خارج Helm محفوظة أثناء الترقية في Helm 3

Helm Components

مقدمة عامة

الجزء ده من الكورس هدفه يعرفك المكونات الأساسية في Helm وإزاي Helm بيشتغل جوه Kubernetes

هيشرح:

- Helm CLI
- Charts (وهي أساس Helm)
- Releases & Revisions
- repositories & ArtifactHub
- templating & values.yaml

◆ ليه بنحتاج Release name ؟

◆ الفرق بين deployment بسيط و deployment معقد

زي WordPress

□ بمعنى آخر : ده يعتبر حجر الأساس اللي لازم تفهمه قبل ما تبدأ تكتب أو تستخدم Charts بنفسك.

مكونات Helm الأساسية (Helm Architecture)

1 أول مكون Helm CLI :

◀ ده البرنامج اللي بتسطبه على جهازك (اللابتوب أو السيرفر اللي بتشتغل منه).
◀ وظيفته:

- تثبيت تطبيق ← **helm install**
 - ترقية ← **helm upgrade**
 - ترجع نسخة ← **helm rollback**
 - تشوف الـ history ← **helm history**
- ⚡ CLI هو واجهتك الأساسية للتعامل مع Helm

2 Charts قلب Helm الحقيقي

◀ الـ Chart هو "Package" التطبيق في Helm
(زي Dockerfile + Kubernetes YAMLs + configuration) كلها في Bundle واحدة.

◀ الـ Chart عبارة عن:

📁 مجموعة ملفات

📄 فيها تعليمات Helm

⚙️ تخلق Kubernetes Objects (Deployment, Service, Ingress, PVC...)

◀ بمعنى آخر:

Chart = مجموعة YAMLs + Templating + values + metadata

Helm بياخد chart ← ويولد منه Kubernetes objects ← وينزلها في الكلاستر.

لما تشغل chart على Kubernetes ، Helm بيعمل "نسخة" منه اسمها : **Release** ★

◆ Release = تثبيت واحد من Chart

◆ كل Release ليه اسم

◆ تقدر تعمل أكثر من Release من نفس الـ chart

مثال : **helm install my-site bitnami/wordpress**

• chart = bitnami/wordpress

• release name = my-site

4 Revisions سجل التعديلات

◀ كل مرة تعمل:

• upgrade - تعديل config - تغيير image - تغيير replicas - rollback

◆ Helm بيعمل Revision جديد.

◆ Revision = Snapshot من حالة التطبيق وقتها

◀ Helm بيحفظ بالـ Revisions دي علشان:

• تعمل Rollback بسهولة

• تعرف حصل إيه في التطبيق

5 Charts Repositories بتتجاب منين ؟

Artifact Hub

◀ نفس الفكرة بتاع Docker Hub .. في Helm عندك:

★ ArtifactHub (helm hub سابقًا)

هتلاقي فيه:

• آلاف Charts جاهزة

• Charts من شركات رسمية (official verified)

• Charts من community

ودا ينفعك إنك:

• تسطب Jenkins ، Prometheus ، Redis ، WordPress

من غير ما تكتب YAMLs بنفسك

6 Metadata — Helm بيخزن معلوماته فين؟

◀ Helm لازم يحتفظ بكل بيانات:

• releases

• revisions

• charts المستخدمة

• تاريخ التغييرات

◀ المعلومات دي اسمها: **Metadata** ★ .. وبتبقى زي السجلات أو logs لكن خاصة بـ Helm

💡 لكن لاحظ ان :

▪ Helm مش بيخزنها على جهازك .. ولكنه بيخزنها جوا Kubernetes نفسه.

▪ فين تحديدا ؟ ← **Kubernetes Secrets** 🌟

■ وده عبقری لیبیین:

1. كل الفريق يقدر يشتغل على Helm من أي جهاز
2. البيانات محفوظة طالما الكلاستر موجود
3. مفيش تعارض بين مستخدمين

7 🌟 نبص بعمق أكثر على مكونات الـ Chart

الـ Chart بيحتوي على ملفات منها:

- templates/ (فيها YAMLs بس بطريقة templating)
- values.yaml ← أهم ملف
- Chart.yaml ← metadata
- charts/ ← لو في dependencies
- files/
- README.md

8 📄 values.yaml — أهم ملف هنتعامل معاه

◀ مهم لأنك من خلاله تقدر :

- تغيير replicas
- تغيير image
- تغيير ports
- تغيير config
- تغيير labels
- تزود env vars

كل دا من values.yaml بس.

◀ 🎯 90% من شغلك هيبقى تعديل values.yaml .. مش تعديل templates

9 🛠️ Templating — ليه بنستخدمه؟

◀ في الـ YAML العادي :

◀ لكن في Helm :

◀ الاختلاف:

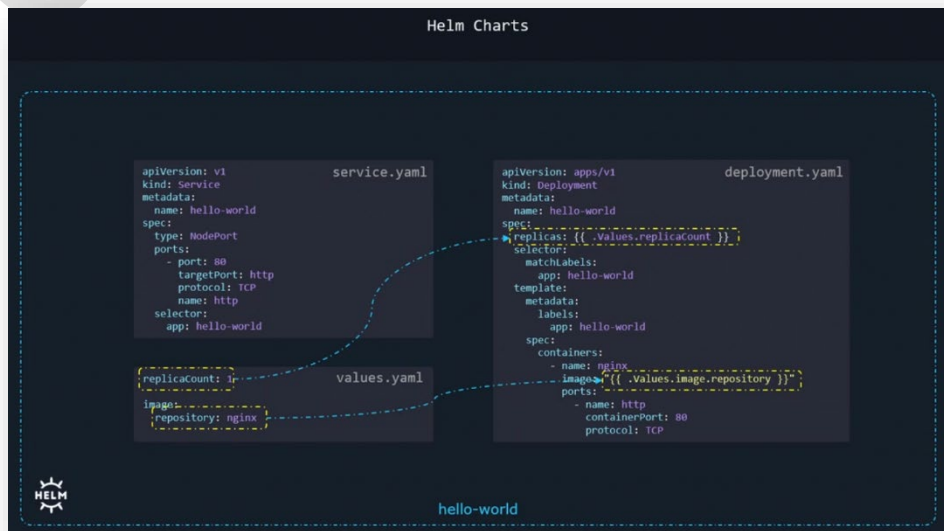
• القيم بقت Dynamic

• وبالتالي هاتغير القيم من values.yaml بدل ما تدخل جوا الـ YAML نفسه

وده مهم جداً في DevOps

```
replicas: 3
image: nginx:latest
```

```
replicas: {{ .Values.replicas }}
image: {{ .Values.image }}
```



هيكون من:

- Deployment
- Service

➤ Helm استخدم علشان الـ image والـ replicas يتنقلوا للـ values.yaml وبالتالي تقدر تخصصهم وقت التثبيت بالشكل ده : **helm install hello ./helloworld --set replicas=5**

WordPress مثال معقد 🚀 1 1

➤ الـ Chart بتاع WordPress ضخم:

- Deployments - StatefulSets
- PVCs
- Secrets
- ConfigMaps
- MariaDB
- Probes
- Ingress
- ServiceAccounts
- RBAC

ومئات الأسطر من templating

وبالتالي ده بيديك فكرة ليه Helm مهم في real production 🚀

🔗 1 2 ليه نحتاج Release Name ؟

➤ ليه مانعملش كده؟ **helm install bitnami/wordpress**

➤ وليه لازم: **helm install mysite bitnami/wordpress**

➤ لأن:

- تقدر تعمل أكثر من نسخة من نفس الـ chart
- كل واحدة منفصلة
- كل واحدة ليها config مختلفة
- كل واحدة ليها lifecycle مختلف

➤ مثال:

- “website-prod”
- “website-dev”

🌐 1 3 وبناء علي النقطة السابقة .. بقا بإمكانك تعمل Deploy Multiple WordPress Sites

تقدر تعمل:

- **helm install my-site bitnami/wordpress**
- **helm install my-second-site bitnami/wordpress**

الـ 2 Releases :

- نفس الـ chart
- نفس الكود
- لكن كيانات مختلفة 100%

• my-site = موقع المستخدمين (Production)

• my-second-site = بيئة التجارب (Dev)

ولما الـ Dev team يضبطوا حاجة ويتأكدوا انها تمام ← ينقلوها للإصدار الأساسي.

آلاف Charts جاهزة 🌐 1 4

من **ArtifactHub** أو Repositories زي:

• Bitnami – TrueCharts – Community Operators - AppCode

وتقدر تستخدم:

- **helm repo add bitnami https://charts.bitnami.com/bitnami**
- **helm search repo wordpress**
- **helm install site1 bitnami/wordpress**

Best Practices 🧠

استخدم values-dev.yaml , values-stage.yaml , values-prod.yaml 🔥

متعدّلش templates إلا لو مضطر 🔥

قبل أي upgrade اعمل : **helm diff upgrade** 🔥

خزن كل قيمك في **Git** 🔥

استخدم release names واضحة 🔥

استخدم verified charts من ArtifactHub لو موجودة 🔥

Real Use Cases 🌱

- نشر WordPress للـ production
- نشر بيئة testing للـ developers
- نشر Redis, Prometheus, Jenkins...
- CI/CD Pipelines بتستخدم Helm للتقريب التلقائي
- إدارة microservices في Kubernetes

ملخص 📄

1. Helm CLI هو اللي بتتعامل معاه.
2. Charts هي أساس نشر التطبيق (Templating + files)
3. لما تثبت Chart ← Helm يعمل Release
4. كل تعديل = Revision جديد.
5. Helm بيخزن metadata كـ **Kubernetes Secrets** علشان فريقك كله يشوفها.
6. values.yaml أهم ملف للتخصيص.
7. تقدر تعمل أكثر من Release من نفس الـ chart
8. معظم Charts الجاهزة موجودة على ArtifactHub
9. Charts بسيطة زي HelloWorld بتبني الأساس، وبعدين WordPress يوضح شغل الإنتاج الحقيقي.

□ الجزء ده ده بيركز بالكامل على Helm Charts :

- بتتكون من إيه؟
- ليه هي أهم جزء في Helm ؟
- بتشتغل إزاي؟
- إزاي الملفات جوا الـ Chart بتتفاعل مع بعضها؟
- يعني إيه templating ؟
- يعني إيه Chart.yaml ؟
- ليه فيه API Version v1 & v2 ؟
- ليه WordPress chart معقد؟
- يعني إيه dependencies ؟
- شكل فولدر الـ Chart نفسه بيبكون عامل إزاي؟

□ الدرس هنا يعتبر العمود الفقري لفهم Helm

ولو فهمته كويس ← هتفهم كل الكورس بسهولة.

1 Automation Tool ← Helm

← Helm مش مجرد tool بتكتب له أوامر... ولكنه automation tool كامل

مثال : لو قولتله مثلا ← **helm install myapp** . 

Helm هايفهم:

- Create Deployment
- Create Service
- Create PVC
- Create ConfigMap
- Assign labels
- Create Ingress
- Apply RBAC rules
- ...إلخ

من غير ما توجّهه لكل خطوة.

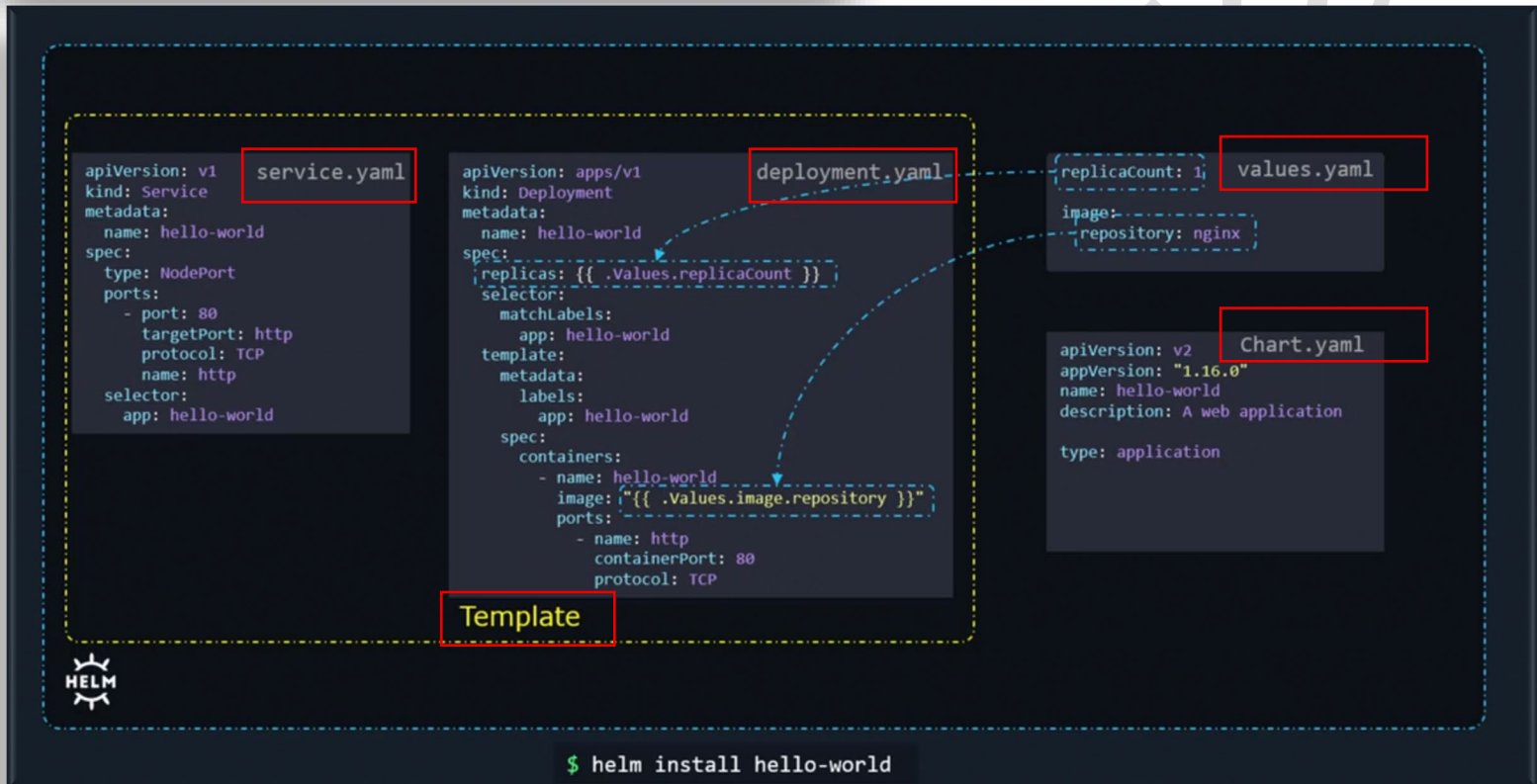
2 طيب هو Helm بيعرف يعمل ده إزاي؟ الجواب Charts :

- Charts = الكتالوج - كتاب التعليمات - وصفة الطبخ - blueprint
- Helm Chart عبارة عن مجموعة ملفات YAML + ملفات Template + ملف Config
- Helm يقرأ الملفات ← يفهم التعليمات ← ينفذهم جوة Kubernetes

الملف	وظيفته
Chart.yaml	تعريف الـ Chart (الاسم - الإصدار - ...dependencies)
values.yaml	الإعدادات التي يستخدمها يحدّلها
/templates	ملفات الـ Kubernetes التي فيها templating
/charts	charts أخرى يعتمد عليها الـ chart
README.md	شرح chart
LICENSE	الرخصة

هندخل في كل جزء بالتفصيل، بس الأول ناخذ overview :

هنشرح كل واحدة بالتفصيل تحت.



values.yaml — المكان الوحيد اللي بتعدّل فيه

لو قتلّك: أنا عايز أغير:

- image
- replicas
- port
- storage size
- database password
- domain name
- resource limits
- إلخ...

هتعمل ده فين؟ values.yaml

وبالتالي فالـ values.yaml ده هو ملف الإعدادات اللي Helm بيستخدمه عشان يبدّل القيم في ملفات الـ templates

اللي بيحصل في الخلفية هو ان Helm بيعمل الآتي :

- ياخذ template ← يقرأ القيم اللي جواه
- يبذلها بقيم من values.yaml
- يخرج ملف YAML النهائي
- بيعته لـ Kubernetes

والعملية دي اسمها **templating**

Templating — السر الحقيقي وراء Helm

❖ Helm templates عبارة عن YAML عادي جدًا

لكن فيه متغيرات زي:

```
replicas: {{ .Values.replicaCount }}
image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

ودي Helm يستبدلها بالقيم اللي في values.yaml

Chart.yaml — بطاقة تعريف الـ Chart

❖ ده الملف اللي بيقول لـ Helm :

“أنا مين؟ أنا إصدار كام؟ أنا بستخدم app version كام؟ عندي dependencies؟”

← أهم الحقول :

1. apiVersion

- Helm 2 ← v1
- Helm 3 ← v2

📌 لو chart مكتوب لـ Helm 2 ← ممكن بيوظ

لو استخدمته على Helm 3

2. version

- ده إصدار الـ chart نفسه .. وبتحدثه لما تغير الملفات أو templating

3. appVersion

- ده إصدار التطبيق الحقيقي .. زي WordPress: 6.4.1 مثلاً.

4. type

- application ← chart عادي لتثبيت تطبيق
- library ← chart بيقدم functions للـ Charts التالية

5. dependencies

- مثال WordPress :
 - WordPress محتاج Database ← فهياضيف MariaDB كـ dependency
 - Helm يتولى:
 - تنزيل MariaDB chart
 - تثبيتها
 - الربط بينها وبين WordPress .. من غير ما تدمج الملفات يدويًا.

```
apiVersion: v2
appVersion: 5.8.1
version: 12.1.27
name: wordpress
description: Web publishing platform for building blogs and websites.
type: application
dependencies:
- condition: mariadb.enabled
  name: mariadb
  repository: https://charts.bitnami.com/bitnami
  version: 9.x.x
  <code hidden>
keywords:
- application
- blog
- wordpress
maintainers:
- email: containers@bitnami.com
  name: Bitnami
home: https://github.com/bitnami/charts/tree/master/bitnami/wordpress
icon: https://bitnami.com/assets/stacks/wordpress/img/wordpress-stack-220x234.png
```

Helm 2

Helm 3

Types

v1

v2

application

library

Artifact Hub

- Helm جمع كل الـ charts من كل شركات العالم في مكان واحد:
- هتلاقي أكثر من 6300 chart جاهزين.
- بعض الشركات اللي بتنشر charts :
 - Bitnami
 - TrueCharts
 - AppsCode
 - Community Operators
 - Prometheus community
- وفيه "official" charts عليها علامة verified

Release — نسخة التطبيق اللي اتسببت بها من chart 8

- لما تعمل : **helm install my-site bitnami/wordpress**
- Helm بينشئ:

- chart ← وصفة
- release ← النسخة المطبقة بالفعل
- 🔗 له release مهم؟

عشان:

- تقدر تنشئ أكثر من Release لنفس التطبيق (زي 2 WordPress websites)
- تقدر تعمل history
- تقدر تعمل rollback
- تقدر تعمل upgrade

Helm – Metadata — بيسجل كل حاجة كـ Secrets 9

❖ Helm لازم يعرف:

- إنت نصبت إيه؟
- إيه هي الإصدارات السابقة؟
- إيه هو آخر config ؟
- إيه اللي اتغير؟

❖ فيخزن ده في Kubernetes كـ Secrets داخل namespace الخاص بالـ release

ده بضمن:

- أي حد في الفريق يشوف history
- metadata تفضل محفوظة حتى لو جهازك اتغير

Chart شكل فولدر الـ 1 0

هيكون عامل كده :

hello-world-chart

- templates # Templates directory
- values.yaml # Configurable values
- Chart.yaml # Chart information
- LICENSE # Chart License
- README.md # Readme file
- charts # Dependency Charts

```

templates/
├── deployment.yaml
├── service.yaml
├── ingress.yaml
└── _helpers.tpl

```

كل حاجة هنا ليها دور مهم.

Best Practices

- استخدم دائما Helm 3
- دائما حذث version في Chart.yaml في حالة انك عدلت الـ chart
- متلعبش في template إلا عند الضرورة
- اعمل override للقيم فقط من values.yaml
- للبيئات المختلفة:

- helm get values <release>
- helm get manifest <release>
- helm history <release>

- values-dev.yaml
- values-stage.yaml
- values-prod.yaml
- استخدم commands دي:

ملخص شامل

1. Helm tool بيشتغل كـ automation engine بياخد هدف وينفذ الخطوات بنفسه.
2. Charts هي "كتاب تعليمات" بيقوله يعمل إيه بالضبط.
3. كل chart فيه ملفات:
 - Chart.yaml ← تعريف
 - values.yaml ← الإعدادات
 - templates/ ← الملفات المطلوب إنشاؤها
4. templating بيحول YAML اللي فيه متغيرات لملفات كاملة جاهزة.
5. releases = التطبيق اللي اتسطب من chart
6. Helm يخزن metadata في Kubernetes كـ Secrets
7. Chart.yaml بقى فيه API version عشان يفرق بين Helm 2 و Helm 3
8. Dependency system بيسمح بدمج charts ثانية بسهولة (زي MariaDB + WordPress)
9. Artifact Hub بيجمع آلاف الـ charts الجاهزة.
10. شكل chart منظم وواضح وتقدر تعدل قيمه بمنتهى السهولة.

ببساطة

Helm = package manager + automation engine + templating system + version control
للـ applications

Working with Helm (Basics)

مقدمة 

□ في الجزء ده هنتعلم إزاي نستخدم Helm CLI على Kubernetes ، وهنشوف:

- البحث عن Charts
 - تثبيت التطبيقات بسهولة.
 - إدارة الـ Releases و التعامل مع Helm Repositories
- الفكرة الأساسية : Helm هو Package Manager للكلاستر، يعني كل التطبيقات والموارد اللي محتاج تثبتها على الكلاستر ممكن تعملها بأمر واحد بس بدل ما تعمل كل حاجة يدويًا.

الخطوات العملية

1 تشغيل Helm CLI

• كل الأوامر في Helm بتتعمل من CLI

• لو كتبت : **helm**

○ هتظهر لك كل الأوامر المتاحة مع شرح بسيط.

❖ لو نسيت أمر معين، استخدم الـ **help** : **helm <command> --help**

❖ لو عايز ترجع Release لحالة سابقة بعد Upgrade فشل،

الأمر هو : **helm rollback <release-name> <revision>**

2 البحث عن Charts

❖ Charts = ملفات YAML منظمة فيها تعليمات تثبيت التطبيق.

❖ طرق البحث:

1. Artifact Hub (موقع ويب)

○ تفتح artifacthub.io وتدور على التطبيق اللي عايزه.

○ اختار Chart عنده Official/Verified Badge لضمان الجودة.

2. من CLI : **helm search hub wordpress**

• hub ← يبحث في Artifact Hub ذات نفسه

• repo ← لو عايز تبحث في Repository معين أضفته قبل كده. (محلياً يعني)

```
$ helm --help

The Kubernetes package manager

Common actions for Helm:

- helm search: search for charts
- helm pull: download a chart to your local directory to view
- helm install: upload the chart to Kubernetes
- helm list: list releases of charts

Usage:
  helm [command]

Available Commands:
  completion generate autocompletion scripts for the specified shell
  create create a new chart with the given name
  dependency manage a chart's dependencies
  env helm client environment information
  get download extended information of a named release
  help Help about any command
  history fetch release history
```

```
$ helm repo --help

This command consists of multiple subcommands to interact with chart repositories.
It can be used to add, remove, list, and index chart repositories.

Usage:
  helm repo [command]

Available Commands:
  add add a chart repository
  index generate an index file given a directory containing packaged charts
  list list chart repositories
  remove remove one or more chart repositories
  update update information of available charts locally from chart repositories

$ helm repo update --help

Update gets the latest information about charts from the respective chart repositories.
Information is cached locally, where it is used by commands like 'helm search'.

Usage:
  helm repo update [flags]

Aliases:
  update, up
```

```
$ helm search wordpress

Search provides the ability to search for Helm charts in the various places
they can be stored including the Artifact Hub and repositories you have added.
Use search subcommands to search different locations for charts.

Usage:
  helm search [command]

Available Commands:
  hub search for charts in the Artifact Hub or your own hub instance
  repo search repositories for a keyword in charts

$ helm search hub wordpress

URL                                CHART VERSION  APP VERSION  DESCRIPTION
https://artifacthub.io/packages/helm/riftbit/wo... 12.1.16        5.8.1        Web publishing platform for building blogs
https://artifacthub.io/packages/helm/bitnami-ak... 12.1.18        5.8.1        Web publishing platform for building blogs
https://artifacthub.io/packages/helm/bitnami/wo... 12.1.27        5.8.1        Web publishing platform for building blogs
```

❖ قبل تثبيت أي Chart من Repo خارجي، لازم تضيف الـ Repo ده :

○ **helm repo add bitnami https://charts.bitnami.com/bitnami**

• bitnami ← اسم الـ Repo المحلي.

• الرابط ← مصدر Charts

🚦 دلوكتي Helm يعرف يجيب Charts من هنا.

4 تثبيت التطبيق

بعد إضافة Repo ، نثبت WordPress على سبيل المثال: **helm install my-release bitnami/wordpress**

• my-release ← اسم Release على الكلاستر.

• bitnami/wordpress ← اسم Chart

العملية دي هتنزل كل الموارد المطلوبة أوتوماتيكياً ← Pods ، Services ، ConfigMaps ... إلخ.

5 إدارة الـ Releases

• كل تثبيت Chart بيكون Release جديد.

• قائمة Releases : **helm list**

• إزالة Release بكل الموارد: **helm uninstall my-release**

🚦 بدل ما تمسح كل حاجة يدوي ، Helm بيعمل كل ده أوتوماتيكياً.

```

Deploying Wordpress

>_

$ helm repo add bitnami https://charts.bitnami.com/bitnami

"bitnami" has been added to your repositories

$ helm install my-release bitnami/wordpress

NAME: my-release
LAST DEPLOYED: Wed Nov 10 18:03:50 2021
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
CHART NAME: wordpress
CHART VERSION: 12.1.27
APP VERSION: 5.8.1

** Please be patient while the chart is being deployed **

Your WordPress site can be accessed through the following DNS name
from within your cluster:

my-release-wordpress.default.svc.cluster.local (port 80)

WordPress

WordPress is one of the most versatile open source content management systems for building blogs and websites.

TL;DR

$ helm repo add bitnami https://charts.bitnami.com/bitnami
$ helm install my-release bitnami/wordpress

USER'S BLOG!
Just another WordPress site

Hello world!

Welcome to WordPress. This is your first post. Edit or delete it, then start writing!

Published May 30, 2021
Categorized as Uncategorized
  
```

6 إدارة Repositories

• عرض Repos موجودة : **helm repo list**

• تحديث معلومات Repos (زي **apt-get update**) : **helm repo update**

🚦 لو Maintainers عدلوا على Charts ، التحديث ده يجدد النسخ المحلية عندك.

Best Practices نصائح

1. استخدم Charts من Verified Publishers دائماً ✅ .

2. لو Upgrade فشل، استخدم **helm rollback** بدل التعديل اليدوي 🔄 .

3. حدث Repos بشكل دوري : **helm repo update**

4. خلي أسماء Releases واضحة عشان تفرق بين التطبيقات 🏷️ .

- ◆ عرض Notes بعد التثبيت : **helm get notes my-release**
 - ◆ معاينة ما سيحدث قبل التنفيذ : **helm install --dry-run --debug ..**
 - ◆ استخراج الـ YAML فقط بدون تنفيذ : **helm template bitnami/wordpress**
-

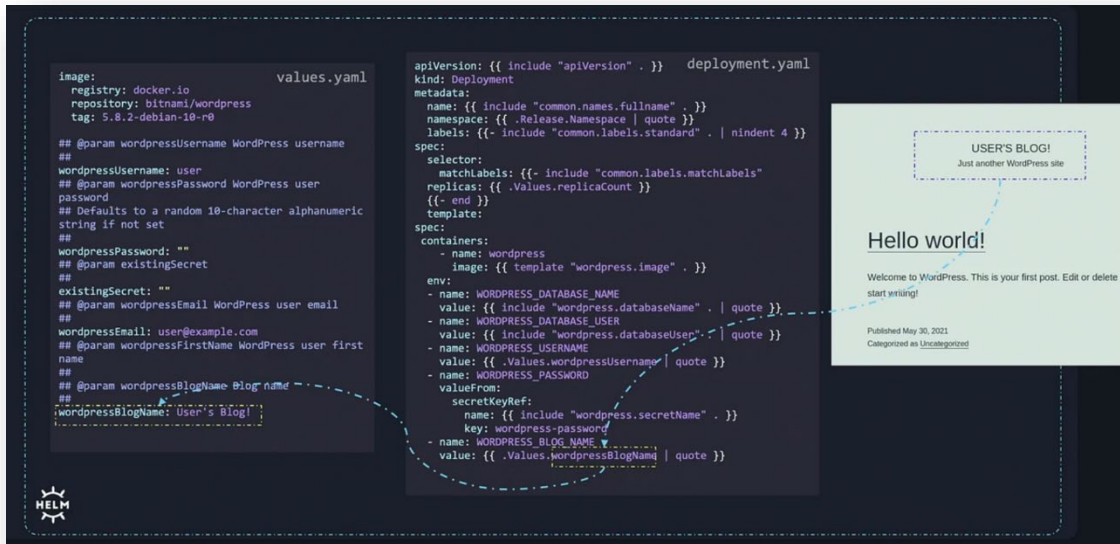
الملخص Summary

- Helm = Package Manager للكلستر.
- Charts = ملفات منظمة بتحدد الموارد المطلوبة للتطبيق.
- CLI = وسيلتك للبحث، التثبيت، إدارة Releases ، وإضافة Repositories
- Helm بيخلي تثبيت، تحديث، وحذف التطبيقات على Kubernetes سهل جدًا.

Customizing Chart Parameters

مقدمة عامة

- لما بنستخدم Helm لتثبيت تطبيق زي WordPress على Kubernetes ، بيكون عندنا قيم افتراضية (values.yaml) محددة مسبقًا.
- لكن في أغلب الحالات، مش عايزين نستخدم القيم دي كما هي، زي اسم ال Blog، ال Email، عدد ال replicas ، أو أي إعداد تاني.



- علشان كده، Helm بيدينا طرق متعددة لتخصيص هذه القيم أثناء التثبيت، من الأبسط للأعقد:

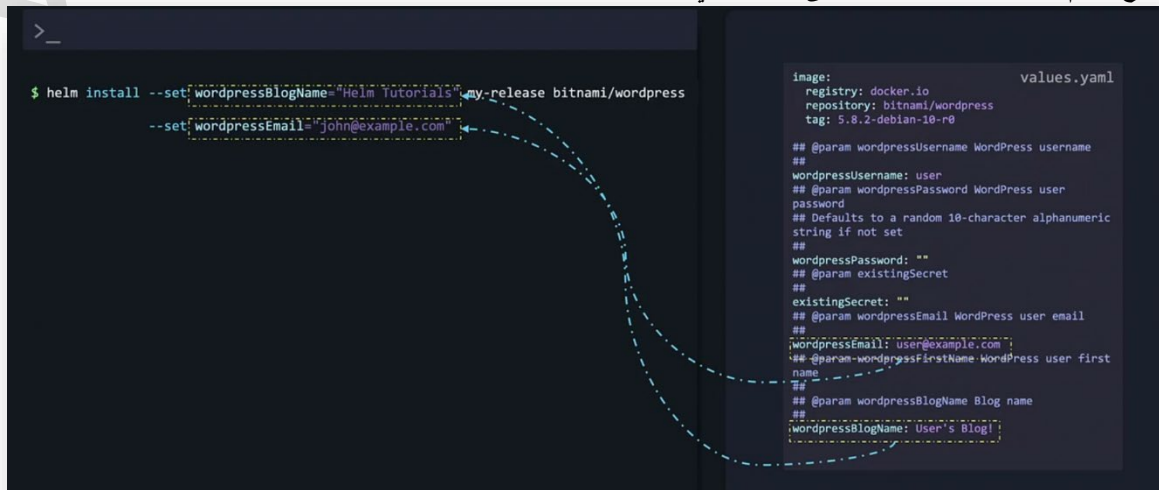
1. التعديل المباشر من CLI باستخدام **--set**
2. استخدام ملف قيم مخصص **--values**
3. تعديل ملف values.yaml الأصلي وتحميل Chart محليًا

شرح خطوة بخطوة

1 التعديل المباشر من CLI ← (--set)

```
helm install my-release bitnami/wordpress \
--set wordpressBlogName="Helm Tutorials" \
--set userEmail="john@example.com"
```

- الفكرة : كل قيمة في values.yaml ممكن نغيرها مباشرة من CLI
- متى نستخدمها : لما يكون عندك عدد قليل من القيم.
- مميزات : سريع، سهل، مش محتاج ملفات إضافية.
- عيوب : مع القيم الكثيرة أو المعقدة، بيبقى غير عملي.



```
# custom-values.yaml
wordpressBlogName: "Helm Tutorials"
userEmail: "john@example.com"
```

1. إنشاء ملف YAML جديد، مثلاً : custom-values.yaml

2. تثبيت Chart باستخدام الملف:

helm install my-release bitnami/wordpress --values custom-values.yaml

- الفكرة : نحتفظ بكل القيم المخصصة في ملف واحد.
- متى نستخدمها : لما يكون عندنا عدد كبير من القيم أو إعدادات معقدة.
- مميزات : منظم، سهل التعديل وإعادة الاستخدام.
- عيوب : لازم تنشئ الملف قبل التثبيت.

```
>_
$ helm install --values custom-values.yaml my-release bitnami/wordpress
```

```
custom-values.yaml
wordpressBlogName: Helm Tutorials
wordpressEmail: john@example.com
```

3 تعديل values.yaml الأصلي

1. تحميل Chart محلياً : **helm pull bitnami/wordpress --untar**
2. تعديل values.yaml : **nano wordpress/values.yaml**
3. تثبيت Chart من المجلد المحلي: **helm install my-release ./wordpress**

- الفكرة : نعدل مباشرة القيم الافتراضية داخل Chart نفسه.
- متى نستخدمها : لما عايزين تغييرات نهائية للنسخة المحلية من Chart
- مميزات : مباشر، لا حاجة لتمرير ملفات إضافية أو CLI
- عيوب : التغيير دائم للنسخة دي من Chart ، مش مرن زي الطرق الثانية.

```
>_
$ helm pull bitnami/wordpress
$ helm pull --untar bitnami/wordpress
$ ls
wordpress
$ ls wordpress
Mode                LastWriteTime         Length Name
----                -
d-----          13-Nov-21 10:36 PM             c1
d-----          13-Nov-21 10:36 PM             templates
-a-----          13-Nov-21 10:36 PM             354 .helmignore
-a-----          13-Nov-21 10:36 PM             399 Chart.lock
-a-----          13-Nov-21 10:36 PM             984 Chart.yaml
-a-----          13-Nov-21 10:36 PM            51819 README.md
-a-----          13-Nov-21 10:36 PM            5918 values.schema.json
-a-----          13-Nov-21 10:36 PM            35737 values.yaml
$ helm install my-release ./wordpress
```

```
values.yaml
image:
  registry: docker.io
  repository: bitnami/wordpress
  tag: 5.8.2-debian-10-r0
# @param wordpressUsername WordPress username
#
wordpressUsername: user
# @param wordpressPassword WordPress user
password
# Defaults to a random 18-character alphanumeric
string if not set
#
wordpressPassword: ""
# @param existingSecret
#
existingSecret: ""
# @param wordpressEmail WordPress user email
#
wordpressEmail: user@example.com
# @param wordpressFirstName WordPress user first
name
#
# @param wordpressBlogName Blog name
#
wordpressBlogName: User's blog!
```

Best Practices ⚡

1. ترتيب الأولويات Override Hierarchy : **CLI (--set) > custom file (--values) > original values.yaml**
2. لو هتغير قيم كثيرة: استخدم custom values file ، هيوفرلك وقت ومجهود.
3. لو عايز نسخة مخصصة تمامًا من Chart : اعمل local values.yaml وتعديل فيها.

ملخص شامل ✓

- Helm Charts ببجوا بقيم افتراضية (values.yaml)
- عند التثبيت ممكن تعدل القيم بطرق مختلفة ← CLI ، ملف مخصص، أو التعديل على values.yaml نفسه.
- كل طريقة ليها مميزات و عيوب حسب حجم وتكرار التغييرات.

- ❑ Lifecycle Management في Helm يعني إدارة دورة حياة التطبيقات على Kubernetes ، من التثبيت (install) ، التحديث (upgrade) ، الرجوع للنسخ السابقة (rollback) ، وحتى الحذف (uninstall)
- ❑ الفكرة ببساطة: كل ما نثبت Chart في الكلاستر، Helm ينشئ Release
- ❖ الـ Release ده عبارة عن مجموعة من الـ Objects (Pods, Services, ConfigMaps...) مرتبطة ببعضها، و Helm بيتابع كل حاجة عنها، وبالتالي يقدر:
 - يحدثها بدون التأثير على Releases تانية
 - يرجعها لأي نسخة سابقة بسهولة

شرح خطوة خطوة

1 تثبيت Release قديمة لتجربة الـ Upgrade

- **helm install nginx-release bitnami/nginx --version 1.19.2**
- هنا اخترنا نسخة قديمة (1.19.2) من NGINX
- Helm أنشأ Release باسم nginx-release يحتوي على Pods و Objects الخاصة بالـ NGINX

```
$ helm install nginx-release bitnami/nginx --version 1.19.2

$ kubectl get pods

NAME                                READY   STATUS    RESTARTS   AGE
nginx-release-687cdd5c75-ztn2n      0/1     ContainerCreating  0          13s

$ kubectl describe pod nginx-release-687cdd5c75-ztn2n

Containers:
  nginx:
    Container ID:  docker://81bb5ad6b5..
    Image:          docker.io/bitnami/nginx:1.19.2-debian-10-r28
    Image ID:       docker-pullable://bitnami/nginx@sha256:2fcdf026b8ac7a..
    Port:           8080/TCP
    Host Port:      0/TCP
    State:          Running
```

2 معرفة نسخة NGINX الحالية

- **kubectl get pods**
- **kubectl describe pod <nginx-pod-name>**
- الهدف: نتأكد أي نسخة شغالة قبل التحديث.
- النتيجة: النسخة 1.19.2.

3 ترقية (Upgrade) Release

- **helm upgrade nginx-release bitnami/nginx**
- Helm حدد كل كائنات الـ Release وقام بتحديثها دفعة واحدة.
- النتيجة: تم إنشاء Pod جديد بـ NGINX 1.21.4 وتم حذف القديم.
- الملاحظة: كل Upgrade يولد Revision جديد، هنا من 1 ← 2.

```
$ helm upgrade nginx-release bitnami/nginx

Release "nginx-release" has been upgraded. Happy Helming!
NAME: nginx-release
LAST DEPLOYED: Mon Nov 15 19:25:55 2021
NAMESPACE: default
STATUS: deployed
REVISION: 2
TEST SUITE: None
NOTES:
CHART NAME: nginx
CHART VERSION: 9.5.13
APP VERSION: 1.21.4

$ kubectl get pods

NAME                                READY   STATUS    RESTARTS   AGE
nginx-release-7b78f4fcd-2zr7k      1/1     Running   0          2m50s

$ kubectl describe pod nginx-release-7b78f4fcd-2zr7k

Containers:
  nginx:
    Container ID:  docker://2a1920aa5409690d9813fe54a3b71..
    Image:          docker.io/bitnami/nginx:1.21.4-debian-10-r0
    Image ID:       docker-pullable://bitnami/nginx@sha256:49080e247d88fae19f..
```

4 متابعة Revisions والـ History

- **helm list**
- **helm history nginx-release**
- **helm list**: يوريك الـ Releases الحالية والـ Revision الحالية.
- **helm history**: تفاصيل كل Revision (chart version ، app version ، action - install/upgrade/rollback)
- ده مهم جداً لو الفريق كبير وكل واحد بيعمل Releases مختلفة.

helm rollback nginx-release 1

- يخلق Revision جديد مشابه للنسخة السابقة.
- مهم : Helm يرجع التعريفات والـ manifests فقط وليس البيانات الحقيقية في الـ Persistent Volumes أو قواعد البيانات.
- للبيانات: لازم تستخدم Chart Hooks أو حلول نسخ احتياطي منفصلة.

ليه أحياناً Helm ما يقدرش يعمل upgrade كامل؟

- ❖ السبب الأساسي: التحديث أحياناً يحتاج صلاحيات أو وصول لموارد خارجية — مثلاً:
 - تطبيق WordPress يحتاج صلاحية لكتابة/تعديل ملفات على الـ filesystem أو الوصول لقاعدة البيانات (لترحيل جداول، تعديل إعدادات... إلخ).
 - Helm بشكل عام يعدل (manifests) وموارد Kubernetes ، لكنه لا يملك بالضرورة بيانات الدخول لقاعدة البيانات أو كلمات السر اللي داخل التطبيق نفسه — فلو التحديث لازم يعمل تغييرات داخل قاعدة البيانات، لازم نعطيها بيانات الدخول أو نستخدم hook يقوم بعمل النسخ/الترحيل.
 - ✚ بالمختصر : لو الترقية تتطلب تغييرات داخل التطبيق أو بيانات خارجية، لازم توفر صلاحيات/آلية للقيام بها — وإلا الترقية هتفشل أو تطلب منك إضافة معلومات/أسرار (secrets)

Helm والـ rollback — إيه اللي بيرجع وإيه اللي مش بيرجع؟

- ❖ Helm يحتفظ بتاريخ الإصدارات للـ Kubernetes manifests (الـ YAML اللي عرفنا بيه الـ Deployments, Services, ConfigMaps، إلخ)
- ❖ لما تعمل helm rollback :
- هيستعيد حالة الكائنات في Kubernetes (مناصب البرمجيات، الإصدارات المستخدمة، تكوين الموارد) إلى الإصدار السابق.
- لكن لو التطبيق أنشأ بيانات داخلية على أقراص متصلة (Persistent Volumes) أو قاعدة بيانات خارجية،
 - Helm مش بيعمل نسخ احتياطي أو يرجع بيانات القاعدة — لأن Helm يتعامل مع التعريفات وليس مع محتوى قواعد البيانات أو الملفات على الأقراص.
- ✚ مثال عملي : لو رجعت MySQL عبر Helm ، الـ Pods والنسخ البرمجية ترجع، لكن الـ data داخل قاعدة MySQL ستظل كما هي و Helm مش هايعدلها.

طيب إيه الحل لو عايز أعمل نسخة من قاعدة البيانات قبل الترقية أو أسترجعها؟

في طريقتين شائعتين:

1. عمل نسخة (dump) للـ DB قبل الترقية — مثلاً mysqldump أو pg_dump تحفظ الملف في مكان آمن (Object Storage أو PVC)
2. استخدام أدوات لعمل نسخ للمخازن (PV snapshots) مثل VolumeSnapshot (لو السحابة أو الـ CSI driver بيدعم) أو أدوات خارجية مثل Velero لعمل backup/restore للـ PV والـ resource YAML
- ✚ لكن الأفضل إننا ندمج خطوة النسخ في عملية الترقية عشان تبقى آلية ومضمونة — وهنا بييجي دور Chart Hooks

❖ الـ Hooks في Helm هي سكربتات / Jobs بتننقذ في مراحل معينة من دورة حياة التثبيت/الترقية/الحذف، مثل:

- **pre-upgrade** ← ينفذ قبل الترقية
 - مثال عملي — شغل Job يعمل mysqldump ويحفظ نسخة احتياطية.
 - **post-upgrade** ← ينفذ بعد الترقية
 - ممكن يتحقق إن الترقية نجحت أو ينفذ مهمة تنظيف.
 - **pre-delete, post-install ... إلخ** ← كل مرحلة ليها hook
- 🌈 بمعنى آخر: لو عندك WordPress + MySQL وتخشى من الفقد، تضيف pre-upgrade hook يبص على DB ويعمل dump، وبعد التحديث لو حصل خطأ تقدر ترجع البيانات يدوياً أو تلقائياً.

أوامر مفيدة في Helm

عمل ترقية مع انتظار نهاية الترقية وإرجاع تلقائي لو فشل

helm upgrade my-release my-chart/ --install --wait --timeout 10m --atomic

- **--atomic** : لو الترقية فشلت، Helm ترجع التغيير تلقائياً للحالة السابقة.
- **--wait** : ببخلي Helm يستنى لغاية ما الـ resources تبقى جاهزة حسب readiness probes

نقاط عملية (Best practices) علشان ترقى بأمان

- جرب أولاً في staging مش مباشرة في production
- سوي Backup للـ DB: dump أو snapshot قبل الترقية.
- استعمل hooks داخل الـ chart لعمل نسخة قبل الترقية (pre-upgrade)
- استخدم secret management لتزويد الـ credentials (Kubernetes Secrets) أو External secrets managers (External secrets managers) بدل ما تضيف كلمات مرور عالـ values.yaml لأنها هاتكون مكشوفة.
- استخدم **--atomic** و **--wait** و **timeout** مناسب لتقليل المخاطر.
- راقب الـ logs والـ readiness probes بعد الترقية للتأكد من سلامة التطبيق.
- لو تحتاج ترجع البيانات بالكامل استخدم أدوات متخصصة (Velero أو snapshots) لأن rollback عادي مش كافٍ.

مثال تطبيقي مبسط (WordPress + MySQL)

□ سيناريو: هعمل ترقية للـ WordPress chart

1. قبل الترقية : pre-upgrade hook يشغل Job يقوم بـ mysqldump ويحفظ الملف على PVC أو S3
 2. أعمل الترقية :
- helm upgrade wordpress-release wordpress-chart/ --install --wait --timeout 10m --atomic**
3. لو الترقية فشلت : Helm يرجع للـ manifests السابقة (لكن الـ DB محتواها ما اتغيرش لأننا حفظناه)
 4. لو محتاج ترجع بيانات DB : استرجع الـ dump اللي أخذناه أو استعمل snapshot

🔥 نصائح عملية Best Practices

1. كل Upgrade يولد Revision جديد، خلي عندك متابعة مستمرة للـ history
2. Rollback مفيد جداً للـ testing أو عند وجود مشاكل بعد التحديث، لكن مش بيرجع البيانات الفعلية للـ DB أو الملفات.
3. قبل أي Upgrade مهم مع تطبيقات تعتمد على DB أو ملفات، اعمل Backup مستقل.
4. استخدم Helm مع Revisions لتتبع كل تغييرات فريقك بشكل منظم.

- كل Chart عند تثبيته يولد Release
- Helm يتابع كل كائن مرتبط بالـ Release ، ويدير lifecycle: Install → Upgrade → Rollback → Uninstall
- التحديث يولد Revision جديد ، Helm يحفظ الـ history لكل نسخة.
- Rollback يرجع التعريفات والمكونات، مش البيانات.
- Helm يجعل إدارة التطبيقات على Kubernetes سريعة، منظمة وآمنة مقارنة بالتحديث اليدوي لكل Pod أو Service
- nginx اختيار عملي لأنه سهل للترقية.
- Helm يعدل تعريفات الـ Kubernetes ويقدر يرجع الإصدارات (rollback) لكن مش مسئول عن محتوى قواعد البيانات أو الملفات في الـ PVs
- لو الترقية تحتاج وصول لقاعدة بيانات أو صلاحيات، لازم تزود Helm/Chart بالـ credentials أو تستخدم hooks تعمل النسخ والترحيل.
- استخدم hooks ، snapshots ، و (best practices (staging, backups, atomic upgrades لتقليل المخاطر.

Writing Helm Charts

مقدمة عامة

- ❑ في الدرس ده هنتكلم عن كتابة Helm Charts من الصفر.
- ❑ الهدف هنا نفهم إزاي نقدر نعمل Chart خاص بينا بدل ما نعتمد على charts جاهزة، وده بيخلينا نتحكم بالكامل في التثبيت، التهيئة، والتخصيص لأي تطبيق Kubernetes
- ❑ الـ Helm Chart ممكن تتشبه بويندوز أو أي نظام تشغيل عنده Installation Wizard : مش بس بيثبت الملفات، لكنه ممكن يظبط إعدادات، يضيف مكتبات، يجهز البيئة، وحتى يعمل نسخ احتياطية قبل التحديث.
- ❑ في البداية، هنتعلم على مثال بسيط : Hello World App باستخدام nginx ، عندنا:

• Deployment بسيط بعدد 2 Replicas

• Service لتوصيل التطبيق

وبعدين هنشوف إزاي نحول ده لـ Helm Chart قابل لإعادة الاستخدام.

الشرح خطوة بخطوة

1 إنشاء هيكل Helm Chart

1. Helm Chart هو مجلد فيه ملفات منظمة بطريقة معينة:

○ Chart.yaml ← معلومات عن الـ chart

○ values.yaml ← القيم الافتراضية

○ templates/ ← ملفات الـ (Deployment, Service, ...) K8S manifest

2. بدل ما نعمل المجلدات والملفات يدوي، نستخدم : **helm create nginx-chart**

○ ده بيعمل Skeleton Chart جاهز للتعديل.

3. نعدل Chart.yaml :

○ نغير الوصف (description)

○ نضيف قسم maintenance مع الإيميل للتواصل

○ ممكن نضيف أي خصائص إضافية زي home, icon, URL

```
service.yaml
apiVersion: v1
kind: Service
metadata:
  name: hello-world
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: http
      protocol: TCP
  selector:
    app: hello-world

deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-world
          image: nginx
          ports:
            - name: http
              containerPort: 80
              protocol: TCP
```

```
hello-world-chart
├── templates
├── values.yaml
├── Chart.yaml
├── LICENSE
└── README.md

$ helm create nginx-chart

$ ls nginx-chart
charts Chart.yaml templates values.yaml
```

```
$ cd nginx-chart
```

```
$ vi Chart.yaml
```



```
Chart.yaml
apiVersion: v2
name: nginx-chart
description: Basic nginx website

#...
type: application

#...
version: 0.1.0

#...
appVersion: "1.16.0"

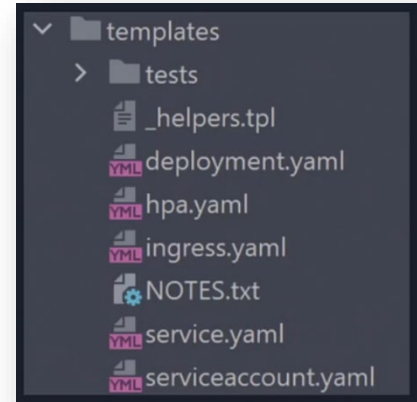
maintainers:
  - email: john@example.com
    name: john smith
```

2 تحضير Templates

1. templates/ ← بيتعمل أوتوماتيك وبيجي فيه ملفات تجريبية من Helm
 - نقدر نحذفها ونشتغل بالملفات اللي عندنا (service و deployment)
2. ننقل ملفات التطبيق (service.yaml و deployment.yaml) لمجلد templates/

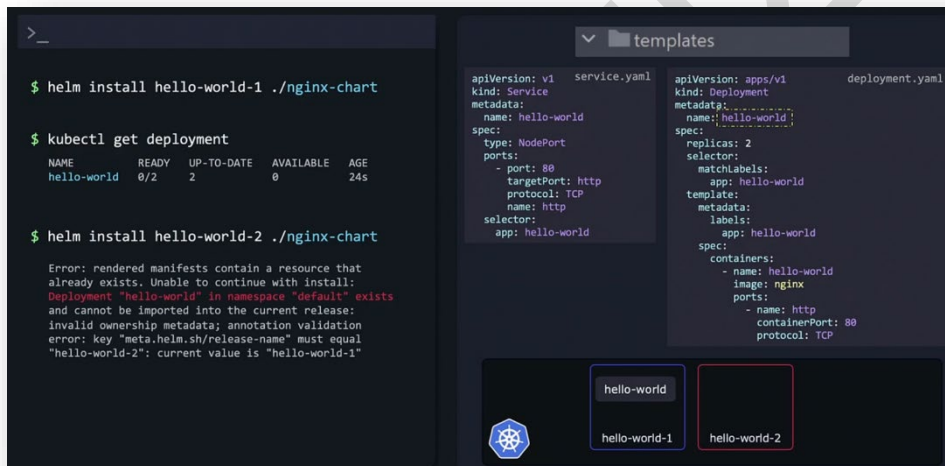
```
$ ls templates
deployment.yaml _helpers.tpl hpa.yaml ingress.yaml
NOTES.txt serviceaccount.yaml service.yaml tests

$ rm -r templates/*
```



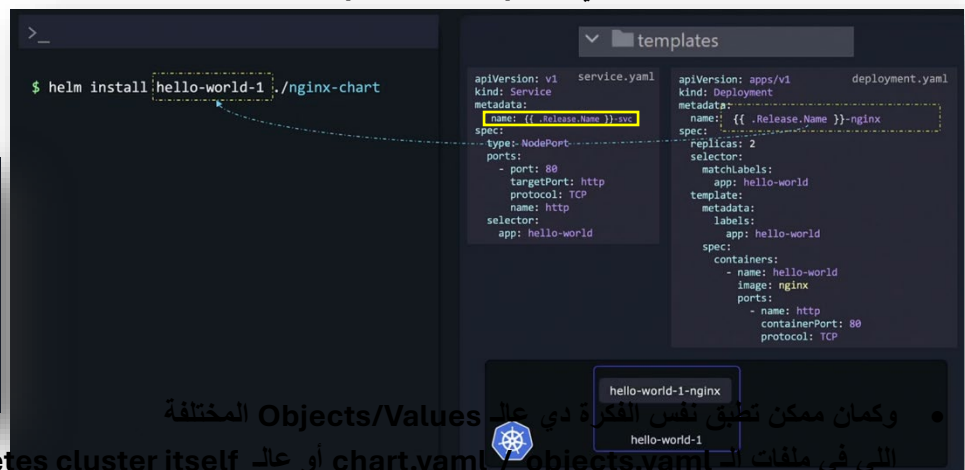
3 مشكلة الأسماء الثابتة (Static Names)

- لاحظ ان الملفات فيها أسماء ثابتة مثل Hello World
 - Release1 ← إنشاء Deployment باسم Hello World
 - Release2 ← محاولة إنشاء Deployment بنفس الاسم ← Error



الحل : استخدام Templating

- Go Template Language → {{ }}
- Release.Name ← الاسم اللي المستخدم اختاره عند تثبيت release
 - ولاحظ ان الـ . ← تشير الي الـ root level or top-level scope



- وكمان ممكن تطبق نفس الفكرة دي على chart.yaml و Objects/Values المختلفة اللي في ملفات الـ chart.yaml أو على Kubernetes cluster itself زي :

Release.Name	Chart.Name	Capabilities.KubeVersion	Capital
Release.Namespace	Chart.ApiVersion	Capabilities.ApiVersions	Small
Release.IsUpgrade	Chart.Version	Capabilities.HelmVersion	Values.replicaCount
Release.IsInstall	Chart.Type	Capabilities.GitCommit	Values.image
Release.Revision	Chart.Keywords	Capabilities.GitTreeState	
Release.Service	Chart.Home	Capabilities.GoVersion	

نقاط مهمة عن Templating

• Helm يعالج بيانات من:

- Release ← بيانات release مثل الاسم ، namespace ...
- Chart ← معلومات chart
- Capabilities ← بيانات الكلاستر
- Values ← القيم المعرفة في values.yaml

• Case Sensitivity :

- Release, Chart, Capabilities ← أول حرف Capital
 - Values ← أول حرف small case (user-defined)
- templating كل الموارد التي محتاجة أسماء فريدة لازم تستخدم

مثال: إذا عملنا:

helm install hello1 ./nginx-chart .. إذا الـ Deployment الأول ← hello1-nginx
helm install hello2 ./nginx-chart .. إذا الـ Deployment الثاني ← hello2-nginx

4 التعامل مع قيم متغيرة من values.yaml

• القيم التي يمكن تخصيصها مثل:

- عدد الـ replicas
- اسم الـ image والإصدار

```
replicaCount: 2
image:
  repository: nginx
  tag: "1.16.0"
pullPolicy: IfNotPresent
```

```
spec:
  replicas: {{ .Values.replicaCount }}
  containers:
    - name: nginx
      image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
      imagePullPolicy: {{ .Values.image.pullPolicy }}
```



```
$ helm install hello-world-1 ./nginx-chart
--set replicaCount=2
--set image=nginx
```

```
# Default values for nginx-chart.
# This is a YAML-formatted file.
# Declare variables to be passed into your templates.

replicaCount: 2
image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: "1.16.0"
```

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-svc
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: http
      protocol: TCP
  selector:
    app: hello-world
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-nginx
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-world
          image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
          ports:
            - name: http
              containerPort: 80
              protocol: TCP
```

أفضل طريقة .. له ؟!

الشرح:
Values.replicaCount ← يسحب القيمة من values.yaml للـ replicas
Values.image.repository ← يسحب اسم الـ image من dictionary
image
Values.image.tag ← يسحب رقم النسخة
Values.image.pullPolicy ← يسحب سياسة سحب الصورة
الميزة: نادر تغير أي قيمة بسهولة في values.yaml أو أثناء التثبيت باستخدام --set

مش أفضل طريقة

أفضل طريقة .. له ؟!

لو كتبنا القيم بدون dictionary :

```
image_repository: nginx
image_tag: 1.16.0
image_pull_policy: IfNotPresent
```

• هنا محتاج تستخدم ثلاث قيم منفصلة في القالب ← تعقيد أكثر
• استخدام (image): dictionary يسمح لنا نصل لكل الخصائص عن طريق Values.image

الشرح:

- replicaCount ← عدد نسخ الـ Deployment
- image ← dictionary (قلموس) يحوي:
- repository ← اسم الـ image
- tag ← رقم النسخة
- pullPolicy ← سياسة سحب الـ image

✓ ملاحظة: استخدام dictionary أفضل من كتابة كل خاصية كقيمة مستقلة لأنه أكثر تنظيم ويسهل التعديل لاحقاً.

• Dictionary Usage :

Values.image.repository

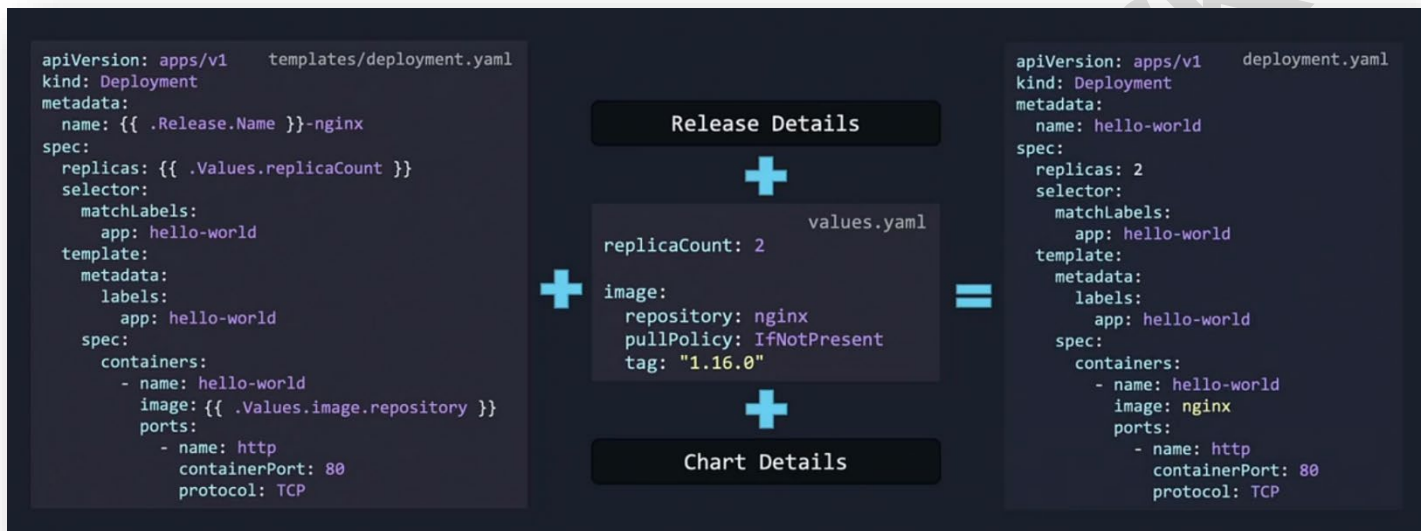
○ Values.image.tag ← إصدار الـ image

✚ باختصار : أي قيمة متغيرة في الـ chart لازم تكون قابلة للتخصيص عن طريق values.yaml أو --set عند التثبيت.

Best Practices & Notes

- تجنب الأسماء الثابتة : أي اسم ممكن يتكرر، اكتبه بالشكل ده {{ .Release.Name }}
- القيم القابلة للتخصيص زي replicas, image, env variables أو أي حاجة ممكن يغيرها المستخدم ← هاتتخط في values.yaml
- استخدام Dictionaries للـ image أفضل من أسماء منفصلة
- Helm Chart يجب أن يكون قابل لإعادة الاستخدام عبر أكثر من release بدون تعارضات أسماء

ملخص شامل



1. Helm Chart = مجموعة ملفات منظمة لإنشاء تطبيق Kubernetes

2. helm create <chart-name> ← ينشئ Skeleton جاهز.

3. Templates تحتاج تخصيص أسماء الموارد لتجنب التعارض بين releases.

4. أي قيمة قابلة للتغيير تستخدم <property>.Values أو --set

5. Helm يستخدم المعلومات من:

○ Release info

○ Chart info

○ Values.yaml

○ Cluster capabilities

لتوليد الملفات النهائية قبل التطبيق على الكلاستر.

النتيجة : Chart جاهز للاستخدام، قابل لتثبيت أكثر من نسخة بدون مشاكل، وقيمته قابلة للتخصيص بسهولة.

Making sure Chart is working as intended

❑ الموضوع هنا عن التحقق من صحة Helm Charts قبل تثبيتها على Kubernetes

❑ في ثلاث طرق رئيسية للتأكد أن الـ Chart شغال صح:

1. Linting — التأكد من تنسيق ملفات YAML والصياغة الصحيحة.
2. Template verification — التأكد أن القوالب (templates) بتترجم للقيم الصحيحة.
3. Dry run — تجربة تثبيت وهمية للتأكد أن الـ Chart يتوافق مع Kubernetes فعليًا.

طريقة استخدام Helm Lint

❖ الهدف: اكتشاف أي أخطاء صياغة أو تنسيق في ملفات الـ Chart

❖ مثال: **helm lint ./mychart**

الشرح:

- Helm يمر على كل الملفات داخل الـ Chart.
- يبحث عن أخطاء مثل:
 - Indentation errors (مشكلة المسافات في YAML)
 - Typos (أخطاء كتابة أسماء المتغيرات، مثل Release بدل .Release)
 - يعطي تقرير مفصل عن: اسم الملف، رقم السطر، ونوع الخطأ.
 - بعد إصلاح الأخطاء وتشغيل الأمر مرة أخرى، يجب أن يظهر: **“0 chart failures”** ✓

💡 نصيحة: Helm Lint كمان يعطي توصيات مثل إضافة أيقونة في Chart.yaml

```

$ helm lint ./nginx-chart

==> Linting ./nginx-chart/
[INFO] Chart.yaml: icon is recommended
[ERROR] templates/: template: nginx-chart/templates/deployment.yaml:4:19: executing "nginx-chart/templates/deployment.yaml" at <.Release.Name>: nil pointer evaluating interface {}.Name

[ERROR] templates/deployment.yaml: unable to parse YAML: error converting YAML to JSON: yaml: line 20: did not find expected '-' indicator

Error: 1 chart(s) linted, 1 chart(s) failed
  
```

After Fixing

```

$ helm lint ./nginx-chart

==> Linting ./nginx-chart/
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed
  
```

The right side of the image shows the content of the templates directory:

```

templates
├── deployment.yaml
├── service.yaml
└── values.yaml
  
```

deployment.yaml content:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-nginx
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-world
          image: {{ .Values.image }}
          ports:
            - name: http
              containerPort: 80
              protocol: TCP
  
```

service.yaml content:

```

apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-svc
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: http
      protocol: TCP
      name: http
  selector:
    app: hello-world
  
```

values.yaml content:

```

# Default values for nginx-chart.
# This is a YAML-formatted file.
# Declare variables to be passed into your templates.

replicaCount: 2
image: nginx
  
```

التحقق من Templates

❖ الهدف: التأكد أن القيم اللي في values.yaml والقوالب تُترجم بشكل صحيح قبل الإرسال لـ Kubernetes

❖ أمر التحقق: **helm template mychart**

الشرح:

- Helm يُنشئ manifests محلياً بدون تثبيت على الكلاستر.
- يعرض كل ما ستنشئه ملفات YAML من القيم المأخوذة من:
 - Values. ← من values.yaml
 - Release.Name. ← اسم الـ release (يمكن تحديده في الأمر ذات نفسه أو يستخدم القيمة الافتراضية)
 - مثال : إذا لم تحدد release name ، سيظهر الاسم الافتراضي RELEASE-NAME
- إذا كان هناك خطأ في التيمبلت (مثل مشكلة المسافات) ، helm template قد يفشل ومش هايوضح مكان الخطأ بالضبط.
- لحل المشكلة : استخدام --debug مع الأمر ليعرض YAML الناتج بالكامل ← **helm template mychart --debug**

The screenshot shows a terminal window with the command `helm template hello-world-1 ./nginx-chart` and its output. The output displays the rendered Kubernetes manifests for a Deployment and a Service. To the right, the source templates are shown: `service.yaml`, `deployment.yaml`, and `values.yaml`. The `values.yaml` file contains default values for `replicaCount` and `image`. A red box highlights the `name` field in the Deployment template, which is populated with `RELEASE-NAME-nginx` from the `values.yaml` file.

⚠ التعامل مع أخطاء Kubernetes

- ❖ هناك أخطاء لا يكتشفها Lint أو Template verification ، لأنها ليست صياغة YAML أو مشكلة Template ، بل أخطاء في Manifest نفسه.

مثال:

```
spec:
  container: # بالجمع "containers" خطأ: يجب أن تكون ✗
```

• Helm lint ✓

• Helm template ✓

• لكن Kubernetes ← يرفض الـ Deployment ✗

- ❖ الحل **helm install myrelease ./mychart --dry-run** : Dry run الشرح:

- يقوم بمحاكاة التثبيت على الكلاستر دون أي تغييرات فعلية.
- يظهر الأخطاء التي سيتوقف عندها Kubernetes عند تثبيت حقيقي.
- بعد إصلاح الأخطاء، يمكنك إعادة dry run للتأكد أن كل شيء مضبوط.

The screenshot shows the output of `helm install hello-world-1 ./nginx-chart --dry-run`. It displays an error message: `Error: unable to build kubernetes objects from release manifest: error validating "": error validating data: [ValidationError(Deployment.spec.template.spec): unknown field "container" in io.k8s.api.core.v1.PodSpec, ValidationError(Deployment.spec.template.spec): missing required field "containers" in io.k8s.api.core.v1.PodSpec]`. This error indicates that the manifest is invalid because it uses the singular `container` field instead of the plural `containers`.

✓ الخلاصة

الهدف

الطريقة

ملاحظات مهمة

يكتشف فقط الأخطاء الصياغية

التحقق من تنسيق YAML وأخطاء الكتابة

helm lint

⚡ نصيحة عملية: دائماً استخدم هذه الخطوات الثلاث قبل أي تثبيت حقيقي على الكلاستر لضمان أن Chart خالي من الأخطاء.

**Template Functions and
Pipelines | Helm**

Function

✿ فكرة المحاضرة

□ الموضوع هنا عن استخدام الدوال (Functions) في Helm

- ❑ الفكرة الأساسية: أحياناً يكون عندك قيمة مش موجودة في values.yaml، وده ممكن يخلي الـ manifest النهائي ناقص ويخلي الـ Pod ما يتنشأش.

```
apiVersion: apps/v1      templates/deployment.yaml
kind: Deployment
metadata:
  name: {{ .Release.Name }}-nginx
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-world
          image: {{ .Values.image.repository }}
          ports:
            - name: http
              containerPort: 80
              protocol: TCP
```

المفروض يبقى مكتوب
هنا nginx

```
values.yaml
replicaCount: 2
image:
  repository:
  pullPolicy: IfNotPresent
  tag: "1.16.0"
```

```
apiVersion: apps/v1      deployment.yaml
kind: Deployment
metadata:
  name: hello-world
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: hello-world
          image:
          ports:
            - name: http
              containerPort: 80
              protocol: TCP
```

وبالتالي
ها يحصل
خطأ هنا

+ =

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-6c76ffbdd7-z4qgf	0/1	InvalidImageName	0	3s

- ❑ الحل: نستخدم دوال لتحديد قيم افتراضية أو لمعالجة النصوص وتحويل البيانات قبل توليد الـ manifest

الدوال بشكل عام

- الدوال في Helm شبيهة بالدوال في لغات البرمجة.
- وظيفة الدالة : تأخذ مدخل (input) وتعطي مخرج (output) بعد معالجة البيانات.
- مثال في البرمجة:

```
upper("helm") → "HELM"
trim(" helm ") → "helm"
```

بنفس الطريقة في Helm ، الدوال بتشتغل على القيم داخل ملفات الـ templates بدون تغيير القيم الأصلية في values.yaml

أمثلة على دوال Helm

1 Upper

- الغرض : تحويل النصوص إلى حروف كبيرة.

```
{{ upper .Values.image.repository }} → image: NGINX
```

- النتيجة: لو repository: nginx سيظهر NGINX في الـ manifest النهائي.

2 Quote

- الغرض: إضافة علامات اقتباس للنصوص.

```
{{ quote .Values.image.repository }} → image: "nginx"
```

- النتيجة: لو repository: nginx سيظهر "nginx" في الـ manifest

3 Replace

- الغرض: استبدال حرف أو نص بآخر.

```
{{ replace "x" "y".Values.image.repository }} → image: "nginy"
```

- يحتوي على 3 مدخلات:

1. النص المراد استبداله (x)
2. النص البديل (y)
3. النص الأصلي (Values.someString).

Shuffle 4

- الغرض: خلط الأحرف في نص ما (نادراً ما يُستخدم، فقط كمثال).

```
{{ shuffle .Values.image.repository }}
```

→ image: "xiggn"

💡 ملاحظة: هناك دوال كثيرة أخرى مثل دوال التشفير، التعامل مع التواريخ، الشبكات، Regex، تحويل أنواع البيانات، إلخ.

Template Function List | Helm

abbrev, abbrevboth, camelcase, cat, contains, hasPrefix, hasSuffix, indent, initials, kebabcase, lower, nindent, nospace, plural, print, printf, println, quote, randAlpha, randAlphaNum, randAscii, randNumeric, repeat, replace, shuffle, snakecase, squote, substr, swapcase, title, trim, trimAll, trimPrefix, trimSuffix, trunc, untile, upper, wrap, wrapWith.

⚡ التعامل مع القيم الافتراضية

- ❖ المشكلة: لو لم يحدد المستخدم قيمة لـ image repository ، لن تظهر في الـ manifest ← Pod يفشل.
- ◀ الحل: استخدام دالة default لتحديد قيمة افتراضية.

<pre>apiVersion: apps/v1 kind: Deployment metadata: name: {{ .Release.Name }}-nginx spec: replicas: {{ .Values.replicaCount }} selector: matchLabels: app: hello-world template: metadata: labels: app: hello-world spec: containers: - name: hello-world image: {{ default "nginx" .Values.image.repository }} ports: - name: http containerPort: 80 protocol: TCP</pre>	+	<pre>values.yaml replicaCount: 2 image: repository: pullPolicy: IfNotPresent tag: "1.16.0"</pre>	=	<pre>apiVersion: apps/v1 kind: Deployment metadata: name: hello-world spec: replicas: 2 selector: matchLabels: app: hello-world template: metadata: labels: app: hello-world spec: containers: - name: hello-world image: nginx ports: - name: http containerPort: 80 protocol: TCP</pre>
---	---	--	---	---

◀ الشرح:

- "nginx" ← القيمة الافتراضية (لازم تكون بين علامات اقتباس لأنها نص).
 - Values.image.repository ← المتغير الذي يمكن للمستخدم تغييره.
 - النتيجة: لو لم يضع المستخدم أي قيمة، الـ manifest النهائي سيحتوي دائماً على nginx
- ✅ مهم: أي قيمة نصية يجب وضعها بين علامات اقتباس، أي قيمة بدون اقتباس تعتبر متغير ويعطي خطأ إذا لم تكن موجودة.

✓ الخلاصة

1. الدوال في Helm تعمل مثل دوال لغات البرمجة: تأخذ مدخل ← تعالج ← تعطي مخرج.
2. يمكن استخدامها لمعالجة النصوص، استبدال الحروف، تحويل الأحرف، إضافة علامات اقتباس، وغيرها.
3. دالة default مهمة جداً لإعطاء قيمة افتراضية في حال لم يحدد المستخدم أي قيمة في values.yaml
4. استخدام الدوال لا يغير القيم الأصلية في values.yaml، بل يغير كيفية ظهورها في الـ manifest النهائي.

Pipelines with Functions

🎯 أولاً: يعني إيه Pipelines أصلاً؟

- ❖ في لينكس لما تعمل: echo "hello" ← النتيجة بتظهر على الشاشة فقط.

❖ طب لو عايز تاخذ نفس الـ o/p وتعدي عليه بعملية ثانية؟ 🤖

هنا بييجي دور الـ **Pipeline** |

مثال: **echo "hello" | tr a-z A-Z** 🔧

- echo تطبع النص
- علامة الـ | بتاخذ الناتج وتبعته كمدخل لأمر ثاني
- tr يحول الأحرف إلى Uppercase

🚩 ده مفهوم الـ Pipeline في لينكس ... ونفس الفكرة موجودة في Helm ! 🥳

🔧 ثانيًا: استخدام الدوال (Functions) في Helm

❖ الطريقة التقليدية لكتابة الدوال : `{{ upper.Values.image.repository }}` يعني:

- الدالة الأول
- المتغير بعدها

🚩 لكن Helm بيديم أسلوب ثاني أقوى وأسهل في القراءة وهو **Pipelines**

🔥 ثالثًا: الـ **Pipelines** في Helm

❖ الطريقة الجديدة : `{{ .Values.image.repository | upper }}` ! نفس النتيجة بالضبط، بس أسهل في القراءة.

💡 ليه الطريقة دي أفضل؟

- بتخليك تبص للبيانات الأول
- وبعدين تشوف سلسلة المعالجات اللي بتحصل عليها
- كأنها "رحلة معالجة" خطوة بخطوة

🚩 رابعًا: تقدر تربط أكثر من **Function** ورا بعض

❖ Helm يسمح لك تعمل كده :

```
{{ .Values.image.repository | upper | quote | shuffle }} ➡ image: GN"XNI"
```

❖ واللي بيحصل:

1 **Values.image.repository** ← تكون مثلاً: nginx

2 **upper** ← النتيجة: NGINX

3 **quote** ← النتيجة: "NGINX"

4 **shuffle** ← يخلط الحروف عشوائيًا .. مثل : "XGNNI"

🚩 كل خطوة بتاخذ ناتج اللي قبلها وتعالجه

🔧 خامسًا: الفرق باختصار

الطريقة باستخدام Pipeline

`{{ Values... | quote | upper. }}`

👉 مقروء وسهل

الطريقة العادية بدون Pipeline

`{{ upper (quote (.Values...)) }}`

👉 شكلها معقد

- Pipeline = نفس فكرة الـ Pipe في لينكس
- بيخلي الكود أسهل في القراءة والكتابة
- تقدر تربط دوال كتير ورا بعض
- كل دالة بتشتغل على إخراج اللي قبلها
- مستخدم جدًا في Helm templates وده اللي هتشوفه في أي Chart محترم

Conditionals

- ❑ Helm templates زي أي لغة برمجة: ساعات محتاج تضيف سطور في الـ YAML لو قيمة معينة موجودة... وساعات لازم متصفهاش لو القيمة مش متعرفة.
- ❑ هنا بييجي دور الـ Conditionals أو الجمل الشرطية.

❖ عندك ملف:

• service.yaml ← جزء من الـ template

• values.yaml ← فيه قيم افتراضية

❖ أنت عايز تضيف Label اسمه org علشان تصنف الـ objects حسب المنظمة.

← المشكلة؟

القيمة دي اختيارية ... ممكن مستخدم الـ chart يحددها وممكن لا.

❗ ولو القيمة مش متعرفة ← مش هاتحتاج ان السطور دي تظهر في الملف النهائي أصلاً

values.yaml

```
replicaCount: 2
image: nginx
orgLabel: payroll
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-nginx
  labels:
    org: payroll
spec:
  ports:
    - port: 80
      name: http
  selector:
    app: hello-world
```

عملنا الـ property ده علشان نقدر
نعمل templating للـ org في ملف
الـ service.yaml

```
labels:
  org: {{ .Values.orgLabel }}
```

الحل : الـ IF Condition في Helm

❖ زي أي كود برمجي، تقدر تقول:

🟢 لو القيمة موجودة ← ضيف السطور

❌ لو مش موجودة ← تجاهلها بالكامل

❖ Helm بيدهم ده باستخدام:

```
orgLabel = "payroll"

if orgLabel:
    print(orgLabel)
end
```

★ اللي بين if و end بيظهر بس لو Values.orgLabel متعرفة.

values.yaml

```
replicaCount: 2
image: nginx
orgLabel: payroll
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-nginx
  {{ if .Values.orgLabel }}
  labels:
    org: {{ .Values.orgLabel }}
  {{ end }}
spec:
  ports:
    - port: 80
      name: http
  selector:
    app: hello-world
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: RELEASE-NAME-nginx
  labels:
    org: payroll
spec:
  ports:
    - port: 80
      name: http
  selector:
    app: hello-world
```

ولو دي مش موجودة

فدي كمان مش هاتبقى
موجودة

🔗 إزاي Helm يتعامل مع المسافات الفاضية؟

❖ Helm لما يزيل الـ template logic بين {{ ... }}

بيسيب أحياناً مسافات فاضية أو أسطر بيضاء في الملف النهائي 🙄

← مثال: لو كتبت:

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-nginx
  {{ if .Values.orgLabel }}
  labels:
    org: {{ .Values.orgLabel }}
  {{ end }}
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: RELEASE-NAME-nginx
  labels:
    org: payroll
```

• Helm هيشيل الجمل نفسها لكن:

○ المسافات قبل {{}}

○ الأسطر اللي كانت موجودة

👤 هتفضل موجودة .. وتطلعك أسطر فاضية زيادة في الـ YAML

🔴 الحل النهائي: قص المسافات باستخدام Dash -

❖ عشان تمنع Helm من ترك المسافات:

⚡ لاحظ:

• الـ Dash بعد {{-}} يمنع المسافة اللي قبل السطر

• الـ Dash قبل {{-}} يمنع المسافة اللي بعد السطر لو عايز

🌟 النتيجة؟

👉 ولا سطر زيادة... ولا مسافات ملهاش لازمة 🤝

```
service.yaml
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-nginx
  {{- if .Values.orgLabel }}
  labels:
    org: {{ .Values.orgLabel }}
  {{- end }}
spec:
  ports:
    - port: 80
      name: http
  selector:
    app: hello-world

service.yaml
apiVersion: v1
kind: Service
metadata:
  name: RELEASE-NAME-nginx
  labels:
    org: payroll
spec:
  ports:
    - port: 80
      name: http
  selector:
    app: hello-world
```

🔴 Helm في IF – ELSEIF – ELSE 🌟

❖ زي أي لغة:

• if

• else if

• else

```
orgLabel = "payroll"

if orgLabel:
    print(orgLabel)
else if orgLabel=="hr":
    print("human resources")
else:
    print("nothing")
end
```

⚡ مثال:

```
service.yaml
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-nginx
  {{- if .Values.orgLabel }}
  labels:
    org: {{ .Values.orgLabel }}
  {{- else if eq .Values.orgLabel "hr" }}
  labels:
    org: human resources
  {{- end }}
```

🔴 ملاحظة مهمة: Helm ما يستخدمش:

• ==

• !=

Function

Purpose

eq

equal

ne

not equal

✂ أشهر استخدام للشرط في Helm ← إنشاء ملفات كاملة

❖ الكثير من الـ charts بيديك اختيار:

✓ هل عايز ServiceAccount ؟

✓ هل عايز ingress ؟

✓ هل عايز PVC ؟

✓ هل عايز Autoscaler ؟

كل ده يتم باستخدام شرط يغطي الملف بالكامل.

← مثال (حرفيًا ملف كامل مش هيتولد إلا بالشرط):

```
values.yaml
# Default values for nginx-chart.
# This is a YAML-formatted file.

serviceAccount:
  # Specifies whether a ServiceAccount should be created
  create: true
```

```
serviceaccount.yaml
{{- if .Values.serviceAccount.create }}
apiVersion: v1
kind: ServiceAccount
metadata:
  name: {{ .Release.Name }}-robot-sa
{{- else }}
```

- لو true = .Values.serviceAccount.create
الـ الملف هيتولد. ✂
- لو false =
✂ الـ الملف كله مش هيتولد في الـ manifest

🔗 الخلاصة

- conditionals في Helm زي البرمجة العادية
- تستخدم if / else / else if
- المقارنات تتم عبر functions زي eq, ne, lt
- استخدم {{- }} و {{- }} لقص المسافات البيضاء
- تقدر تتحكم في ظهور سطور، أو بلوكات، أو ملفات كاملة

With Blocks

🔗 مقدمة عامة

❑ الـ Scope ببساطة هو:

“أنا واقف فين دلوقتي؟ ومن النقطة دي أقدر أشوف/أوصل لإيه؟”

❑ Helm templates فيها نظام مستويات (Hierarchy) للقيم، وكل مستوى اسمه Scope

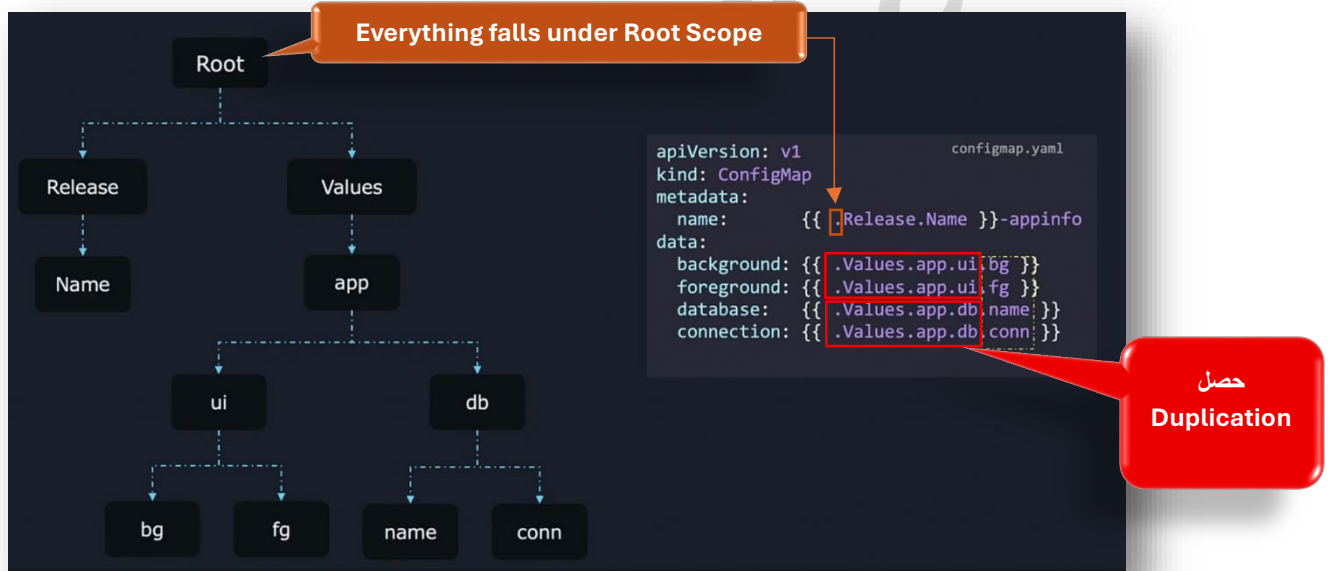
❖ عندك values.yaml فيه:

```
app:
  ui:
    bg: red
    fg: black
  db:
    name: "users"
    conn: "mongodb://localhost:27020/mydb"
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-appinfo
data:
  background: {{ .Values.app.ui.bg }}
  foreground: {{ .Values.app.ui.fg }}
  database: {{ .Values.app.db.name }}
  connection: {{ .Values.app.db.conn }}
```

❖ وعندك Template اللي هو ← configmap.yaml .. وعازب تقرأ القيم دي منه .. فالسؤال بقا هو ازاي !!؟!

❖ يعني إيه "." Dot ؟



❖ النقطة . ← معناها الـ Scope الحالي

• ولو في بداية أي ملف Template ← الـ Dot معناها الـ Root

❖ وعلشان توصل لأي قيمة لازم تبدأ من أول الـ Hierarchy (اللي هو الـ Root) لغاية الـ Object اللي انت عاوزه (مثلا fg)

• وده يؤدي لتكرار رهيب... وملف شكله ثقيل ومكرر 🤖

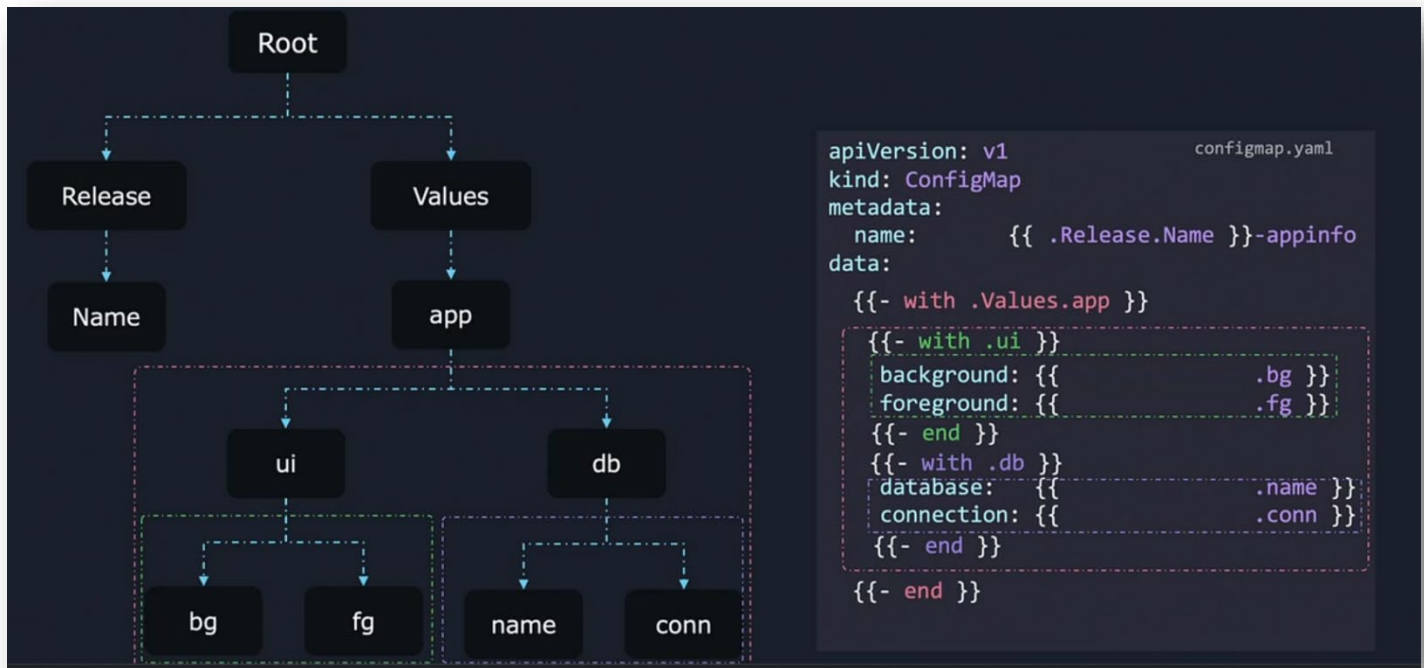
🔧 الحل: استخدام الـ Nested WITH block لتغيير الـ Scope وإنشاء scopes داخل الـ scopes

❖ بدل ما تكتب كل مرة:

```
.Values.app.ui.BG
.Values.app.ui.FG
.Values.app.db.name
```

❖ ممكن نقول لـ Helm :

“اعتبر إن الـ Scope دلوقتي هو “Values.app”
مثال:



لا اخط ان باستخدام كونسبيت الـ
Nested Scope فانت ممكن تعمل:
✓ تغيير الـ Scope لـ app
✓ ثم تغييره مرة أخرى لـ ui

- الـ Dot داخل الـ with الأولي بقى = `.Values.app`
وبالتالي المفروض كان يبقي :
○ `.ui.BG = Values.app.ui.BG`
○ `.ui.FG = Values.app.ui.FG`

لكن احنا استغلينا كونسبيت الـ Nested Scope وعملنا `Scope {{- with .ui}}` داخل الـ `scope {{- with .values.app}}`

- `Dot = .Values.app.ui`
فتقدر تكتب `.BG` و `.FG` ببساطة
- أو `Dot = .Values.app.db`
فتكتب `.name` و `.connection`

وبالتالي أصبح أنيق جدًا ومختصر 🥳

⚠️ المشكلة المشهورة: لما تفقد الوصول لقيم من الـ Root

❖ لو حاولت داخل الـ with تقول: `release: {{ .Release.Name }}`
هتاخذ Error :

nil pointer evaluating release.name

ليه؟ 🤔

لأن الـ Dot بقى `.Values.app` (يعني بيشاور عالـ scope ده)
و ملهاش حاجة اسمها `.Release`.

🔧 نوعا ما نفس الفكرة في لينكس لو انت جوا `dir` معين وجيت تعرض الـ `files` .. هاعرضلك الـ `files` اللي جوا الـ `dir` ده فقط

💡 الحل الذهبي: استخدام الـ `Dollar $`

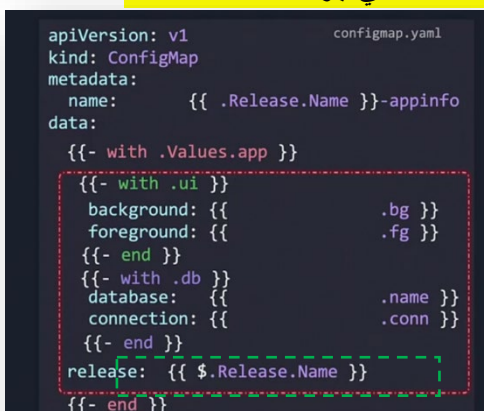
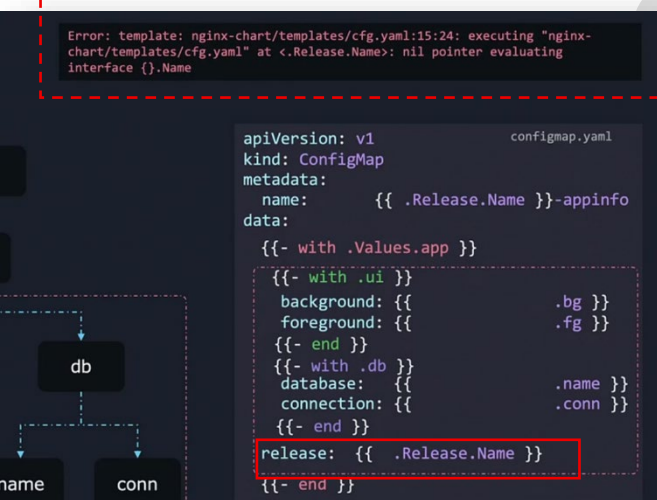
❖ الرمز `$` معناه :

ارجع للـ `Root scope` مهما كنت واقف فين

➔ يعني : `release: {{ $.Release.Name }}`

▪ ده يوصلك للـ `Release` حتى لو جوة 10 `WITH blocks`.

🧠 خريطة ذهنية (ملخص سريع)



الرمز	معناه
.	ال Scope الحالي
{{ with X }}	غير ال Scope إلى X
{{ end }}	ارجع لل Scope السابق
\$	ارجع لل Root scope

🔗 الخلاصة

- Dot = المكان الذي Helm واقف فيه
- WITH = طريقة تغيّر بيها مكان الوقوف
- Nested WITH = مستويات داخل مستويات
- \$ = الطريق السريع لل Root
- Scope يساعدك تقلل التكرار وتخلي ال templates أنضف

Loops & Ranges

🔥 الفكرة العامة

- ❑ في أي لغة برمجة، (for loop) هو الذي يخليك تكرر نفس الكود أكثر من مرة على حسب البيانات التي عندك.
- ❑ نفس الكلام في Helm، بس بدل ما نكتب كود برمجة، نكتب Go templates

1 يعني إيه Loop في Helm ؟

```
regions: values.yaml
- ohio
```

```
configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: RELEASE-NAME-regioninfo
data:
```

❖ عندك list (مثلاً list من الـ Regions)

- ❖ فلو عاوز تعمل ConfigMap وعاوز تكتب فيه الـ regions دي كـ Array
- فبدل ما تكتبهم manual .. نستخدم range

2 نبدأ بالـ ConfigMap الأساسي

```
configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-regioninfo
data:
  regions:
```

- ❖ لحد هنا تمام .. إحنا بس عاملين الهيكل، ولسه عاوزين نضيف اللست.

3 نعمل Loop باستخدام range

- ❖ في Helm بنكتب اللوب كده :

- اللي جوا اللوب بيتكرر مرة لكل item في regions

4 نفهم الـ Scope

- ❖ دي أهم فكرة 🙌

بما ان كل الـ regions بتبدأ بـ - .. فإحنا حطينا - في الـ range block

جوه range :

- النقطة {{ . }} مش بتمثل كل values

ولكنها بتمثل العنصر الحالي في اللست

○ يعني أول "Iteration = "ohio

○ تاني "Iteration = "virginia" وهكذا

🧑‍💻 حاجة كذا زي foreach(item in list) في البرمجة.

لاحظ انك لو عملت كذا فهايطبعك الـ regions من غير الـ "" وعلشان نحل المشكلة دي هانضيف الـ quotes

5 نكتب الشكل اللي

- ❖ يعني نكتب inside range :

➤ فيه استخدمنا | quote ؟

عشان يعمل "" حوالين القيمة تلقائيًا .

```
configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-regioninfo
data:
  regions:
    {{- range .Values.regions }}
    - {{ | quote }}
    {{- end }}
```

❖ Helm هيطلع:

بالظبط اللي إحنا عاوزينه 100

```

apiVersion: v1                                configmap.yaml
kind: ConfigMap
metadata:
  name: RELEASE-NAME-regioninfo
data:
  regions:
    - "ohio"
    - "newyork"
    - "ontario"
    - "london"
    - "singapore"
    - "mumbai"

```

📁 الخلاصة

- اللوب في Helm = range
- جوه الـ range العلامة . بنساوي العنصر الحالي في اللست
- علشان أضيف Quotes ← أستخدم | quote
- أي object (حتى dot نفسها) تقدر تعمل عليه Pipelines
- الناتج : List اتبنت dynamically من values.yaml بدون ما تكتب حاجة manual

Named Templates

🔗 Helm في Named Templates & Helpers

1 المشكلة الأساسية

- ❑ في ملفات الـ Kubernetes (Deployment ، Service ، ... إلخ) غالبًا بنلاقي خطوات متكررة كتير زي :

```

apiVersion: apps/v1                            deployment.yaml
kind: Deployment
metadata:
  name: {{ .Release.Name }}-nginx
labels:
  app.kubernetes.io/name: nginx
  app.kubernetes.io/instance: nginx
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: nginx
      app.kubernetes.io/instance: nginx
  template:
    metadata:
      labels:
        app.kubernetes.io/name: nginx
        app.kubernetes.io/instance: nginx
    spec:
      containers:
        - name: nginx

```

- الخطوات دي ممكن تتكرر في Deployment و Service وأي objects تانية.
- لو عدلت السطور دي يدوي، ممكن يحصل خطأ أو تناقض.
- ❑ الحل؟ نستخدم Named Templates علشان نحقق مبدأي :

Reuse Code
Remove Duplication

2 إنشاء Helper Template

- ❖ الخطوة الأولى: ملف helpers.tpl
 - أي ملف يبدأ بـ `{}- define` بيتم تجاهله
 - ده بيخلينا نحط فيه وظائف مس
- مثال:

```

{{- define "mychart.labels" -}}
labels:
  app: {{ .Release.Name }}
  release: {{ .Release.Name }}
  env: {{ .Values.env }}
{{- end -}}

```

Kubernetes manifest

حطينا هنا labels علشان
دا الجزء المتكرر في سكشن
الـ labels

```

{{- define "labels" -}}
  app.kubernetes.io/name: nginx
  app.kubernetes.io/instance: nginx
{{- end -}}

```

المشكلة هنا ان الـ Release name → hardcoded
وبالتالي دا هيعمل مشكلة لو عملنا install multiple releases

define ← بتدي الاسم للـ template المساعد

```

apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-nginx
  labels:
    {{- template "labels" -}}
spec:
  ports:
    - port: 80
      targetPort: http
      protocol: TCP
      name: http
  selector:
    app: hello-world

```

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-release-nginx
  labels:
    app.kubernetes.io/name: nginx
    app.kubernetes.io/instance: nginx
spec:
  ports:
    - port: 80
      targetPort: http
      protocol: TCP
      name: http
  selector:
    app: hello-world

```

المشكلة هنا ان الـ scope بيبقي موجود في الـ template files فقط ومش
بيكون موجود في الـ helper file (يعني الـ helper file مش بيشوفه)
وبالتالي فالـ helper file مش هيلقي داتا يتعامل معاها !!

```

{{- define "labels" -}}
  app.kubernetes.io/name: {{ .Release.Name }}
  app.kubernetes.io/instance: {{ .Release.Name }}
{{- end -}}

```

الحل هو اننا هانضيف Template Directive وبالتالي استخدمنا
hardcode بدل .Values.env و .Release.Name

```

apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-nginx
  labels:
    {{- template "labels" . -}}
spec:
  ports:
    - port: 80
      targetPort: http
      protocol: TCP
      name: http
  selector:
    app: hello-world

```

الحل هو اننا هانضيف . في الـ template file
علشان تمرر الـ current scope للـ helper
لأن بدونها، Release.Name مش هيتعرف عليه

❖ في أي مكان في Deployment أو Service دي هاتبقي طريقة الاستدعاء النهائية عموما :

❖ النقاط المهمة: (الشرح والتوضيح)

1. | indent 2 or 4 :

- يظبط الـ indentation حسب المكان المطلوب
- (يعني بيظبط المسافة الفاصلة اللي قبل الكلام لأن yml عموما حساس للحاجات دي)
- كل instance ممكن تحتاج عدد spaces مختلف

```

matchLabels:
  {{- include "labels" . | indent 2 }}

```

2. Include :

- يسمح لنا نستدعي template helper
- على عكس template، نقدر نعمل له pipe لأوامر تانية زي indent

```

_helpers.tpl
{{- define "labels" }}
  app.kubernetes.io/name: {{ .Release.Name }}
  app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

```

```

deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-nginx
  labels:
    {{- template "labels" . }}
spec:
  selector:
    matchLabels:
      {{- include "labels" . | indent 2 }}
  template:
    metadata:
      labels:
        {{- include "labels" . | indent 4 }}
    spec:
      containers:
        - name: nginx
          image: "nginx:1.16.0"
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 80
              protocol: TCP

```

```

deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: RELEASE-NAME-nginx
  labels:
    app.kubernetes.io/name: nginx-chart
    app.kubernetes.io/instance: nginx-release
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: nginx-chart
      app.kubernetes.io/instance: nginx-release
  template:
    metadata:
      labels:
        app.kubernetes.io/name: nginx-chart
        app.kubernetes.io/instance: nginx-release
    spec:
      containers:
        - name: nginx
          image: "nginx:1.16.0"
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 80
              protocol: TCP

```

إكتفيت هنا بال template فقط علشان مش محتاج تعديلات عال o/p

عملت هنا include علشان محتاج أظبط الـ indentation وأعمل pipe

include	template	الخاصية
نعم ✓	لا ✗	يعمل pipe ؟
نعم ✓	نعم ✓	يستخدم helper ؟
تمرير dot لازم برضه	تحتاج تمرير dot	يمرر scope ؟

4 الفرق بين template و include

5 مثال كامل في Deployment

- الـ labels اتركروا صح بدون كتابة lines كثير
- كل instance متنسقة بالـ indentation المناسب

```

_helpers.tpl
{{- define "labels" }}
  app.kubernetes.io/name: {{ .Release.Name }}
  app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

```

```

deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-nginx
  labels:
    {{- template "labels" . }}
spec:
  selector:
    matchLabels:
      {{- include "labels" . | indent 2 }}
  template:
    metadata:
      labels:
        {{- include "labels" . | indent 4 }}
    spec:
      containers:
        - name: nginx
          image: "nginx:1.16.0"
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 80
              protocol: TCP

```

```

deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: RELEASE-NAME-nginx
  labels:
    app.kubernetes.io/name: nginx-chart
    app.kubernetes.io/instance: nginx-release
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: nginx-chart
      app.kubernetes.io/instance: nginx-release
  template:
    metadata:
      labels:
        app.kubernetes.io/name: nginx-chart
        app.kubernetes.io/instance: nginx-release
    spec:
      containers:
        - name: nginx
          image: "nginx:1.16.0"
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 80
              protocol: TCP

```

template

action

include

function

الخلاصة

- Named Templates تحل مشكلة التكرار في Helm
- _helpers.tpl ← مكان للـ helpers مش هيتحول لـ manifest
- define ← يعطي اسم للـ helper
- + indent + include ← استدعاء helper مع تمرير scope وضبط المسافات.
- هتخلي ملفاتك نظيفة، متناسقة، وسهلة الصيانة.

Chart Hooks

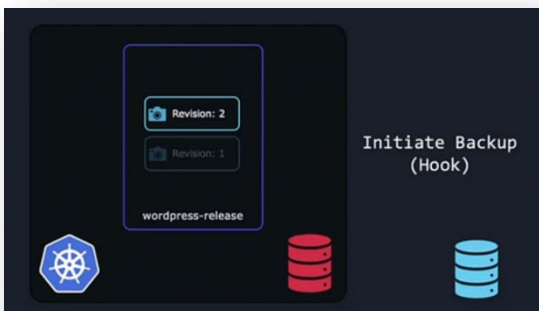
1 مقدمة عن الـ Helm Hooks

❑ الـ Helm هو أداة لإدارة التطبيقات على Kubernetes من خلال ما يُسمى Charts، التي هي عبارة عن مجموعة ملفات جاهزة لتنصيب التطبيق.

❑ عادة، الـ Helm يثبت الموارد (Pods, Services, ConfigMaps إلخ) مباشرةً.

❖ لكن أحياناً نحتاج أن نفعل أشياء إضافية قبل أو بعد تثبيت التطبيق، مثل:

- أخذ نسخة احتياطية من قاعدة البيانات قبل التحديث.
- إرسال بريد إلكتروني قبل التنصيب.
- عرض رسالة على الموقع أثناء الترقية ثم إزالتها بعد انتهاء العملية.



2 دورة حياة ال Helm وعلاقة ال Hooks

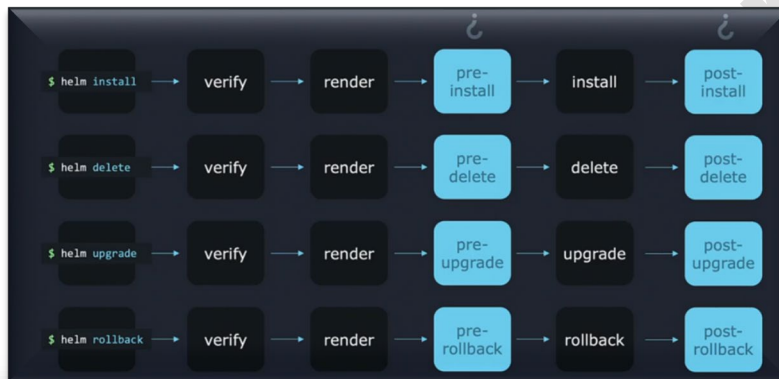
❖ دورة الأحداث الأساسية عند تنفيذ:

- لتنصيب التطبيق : **helm install <chart>**
- لترقية التطبيق : **helm upgrade <chart>**

تكون كالتالي:

1. تحقق Helm من الملفات والقوالب (templates)
2. تحويل القوالب إلى ملفات Kubernetes جاهزة للتنفيذ.
3. تطبيق الملفات على الكلاستر (install/upgrade phase)

❖ ال Hooks تتيح تنفيذ أوامر قبل أو بعد هذه المراحل، على سبيل المثال:



نوع ال Hook	متى يتم التنفيذ	مثال
pre-install	قبل تثبيت ال chart لأول مرة	إعداد قاعدة البيانات
post-install	بعد التثبيت	إرسال إشعار نجاح
pre-upgrade	قبل الترقية	backup للبيانات
post-upgrade	بعد الترقية	تنظيف ملفات مؤقتة
pre-delete	قبل الحذف	إرسال تحذير
post-delete	بعد الحذف	تنظيف موارد إضافية
pre-rollback	قبل التراجع	تسجيل الحالة
post-rollback	بعد التراجع	إعادة تهيئة البيئة

3 كيف نربط Hook بعملية في Kubernetes

❖ نفترض عندنا سكربت **backup.sh** لأخذ نسخة احتياطية.. ونريد أن يُنفذ مرة واحدة قبل الترقية.

- Kubernetes Pod : يعمل بشكل مستمر (forever) وبالتالي هو مناسب للتطبيقات، لكنه غير مناسب للسكربت التي هابشتغل مرة واحدة فقط.
- Kubernetes Job : يُنفذ مرة واحدة وينتهي وبالتالي فهو مناسب لل Hooks مثل نسخة احتياطية.

❖ يعني لإضافة Hook ، نكتب Kubernetes Job داخل مجلد templates في Helm Chart

- ❖ لكي يعتبر Helm الملف كـ Hook وليس كـ Template عادي، نستخدم Annotations :
- وظيفة الـ Annotations ده ان يضيف metadata زيادة للـ object اللي اليوزر هاستخدمه (في حالتنا دي هابقي الـ Helm) .. وهايحط في الـ metadata دي معلومات عن الـ object وهكذا

مثال : Job تعمل backup

- "helm.sh/hook": pre-upgrade ← يخبر Helm أن هذا Job يجب أن يُنفذ قبل الترقية.
- يمكن تغيير القيمة إلى pre-install, post-install, post-upgrade حسب الحاجة.

```

apiVersion: batch/v1
kind: Job
metadata:
  name: {{ .Release.Name }}-nginx
  annotations:
    "helm.sh/hook": pre-upgrade
spec:
  template:
    metadata:
      name: {{ .Release.Name }}-nginx
    spec:
      restartPolicy: Never
      containers:
        - name: pre-upgrade-backup-job
          image: "alpine"
          command: ["/bin/backup.sh"]

```

hello-world-chart

templates

- service.yaml
- deployment.yaml
- secret.yaml
- backup-job.yaml

pre-upgrade

Job

backup.sh

5 ترتيب تنفيذ الـ Hooks

- ❖ في بعض الحالات عندنا أكثر من Hook لنفس المرحلة (مثلاً عدة pre-upgrade jobs) في الحالة دي نستخدم weights لتحديد ترتيب التنفيذ:
- ❖ الترتيب:

```

annotations:
  "helm.sh/hook": pre-upgrade
  "helm.sh/hook-weight": "5"

```

1. تصاعدياً حسب weight
2. نفس weight ← حسب resource kind
3. نفس kind ← حسب الاسم ascending

\$ helm upgrade

verify

render

pre-upgrade

upgrade

Email Announcement

Setup Banner

Backup database

Weights

-4

3

5

```

apiVersion: batch/v1
kind: Job
metadata:
  name: {{ .Release.Name }}-nginx
  annotations:
    "helm.sh/hook": pre-upgrade
    "helm.sh/hook-weight": "5"
spec:
  template:
    metadata:
      name: {{ .Release.Name }}-nginx
    spec:
      restartPolicy: Never
      containers:
        - name: pre-upgrade-backup-job
          image: "alpine"
          command: ["/bin/backup.sh"]

```

6 تنظيف الموارد بعد تنفيذ الـ Hook

- بعد تنفيذ الـ hook ، الموارد (سواء كانت Job أو Pod) بتفضل موجودة.
- نستطيع التحكم بسلوك الحذف عبر hook deletion policy :

سياسة الحذف	النتيجة
hook-succeeded	يحذف المورد بعد التنفيذ الناجح
hook-failed	يحذف المورد حتى لو فشل التنفيذ
before-hook-creation (default)	يحذف المورد القديم قبل إنشاء hook جديد

- هذه السياسات تمنع إنشاء duplicate objects بنفس الاسم في Kubernetes مثال:
هذا يمنع تكرار Job أو ظهور أخطاء عند محاولة إنشاء موارد بنفس الاسم.

7 الخلاصة

- Chart Hooks = طريقة لتنفيذ إجراءات إضافية قبل/بعد install, upgrade, delete, rollback
- يتم تعريف الـ hook كـ Kubernetes object مع annotation helm.sh/hook
- ترتيب التنفيذ يتحكم فيه hook weight
- تنظيف الموارد بعد التنفيذ يتحكم فيه hook deletion policy

Packaging and Signing Charts

1 تجهيز Chart للنشر

- ☐ قبل ما نبدأ نرفع أو نشارك Chart مع أي حد، لازم نجهزه.
- ☐ الـ Chart عبارة عن مجموعة ملفات منظمة بطريقة معينة:
- ❖ مكونات الـ Chart :

1. Chart.yaml ← يحتوي على معلومات أساسية عن الـ chart : الاسم، النسخة، الوصف، المؤلف.
2. values.yaml ← القيم الافتراضية للـ variables اللي هتستخدم في الـ templates
3. templates/ ← ملفات الـ Kubernetes YAML اللي Helm هيحولها لـ manifests
4. charts/ ← لو الـ Chart بيعتمد على Charts تانية (Dependencies)
5. README.md ← دليل مختصر عن الـ chart وطريقة الاستخدام.
6. License files ← ترخيص الاستخدام.

```
$ ls nginx-chart
charts Chart.yaml templates values.yaml README.md
LICENSE
```

: Packaging Chart

- الأمر الأساسي: **helm package nginx**
- النتيجة: ملف مضغوط باسم **nginx-0.1.0.tgz**
- 0.1.0 ← النسخة من **Chart.yaml**
- tar + gzip (ملف مضغوط) ← **.tgz**

إليه الذي جوه الملف المضغوط؟

- كل الملفات التي احنا حطيناها في مجلد **nginx** وبشكل مرتب وسهل تحركه أو تشاركه

ملاحظات:

- يمكنك فتح **.tgz** بأي Archive manager مثل WinRAR على ويندوز أو **tar -tzf nginx-0.1.0.tgz** على Linux

2 أهمية توقيع Chart

❖ لو مجرد رفعنا **.tgz** على الإنترنت:

- ممكن يتعرض للتغيير من هاكلز
- ممكن المستخدمين يثبتوا chart غير أصلي

الحل: توقيع Chart رقمياً باستخدام GPG

❖ ماذا يقدم التوقيع؟

- يضمن أن الملف الذي المستخدم نزل بالضغط نفس الملف الذي أنت رفعته
- إذا حدث أي تغيير ← التحقق هيفشل

3 توليد مفاتيح GPG

❖ Helm يحتاج مفتاح خاص Private key للتوقيع

و مفتاح عام Public key للتحقق.

❖ خطوات توليد المفتاح بسرعة (for exercise)

gpg --quick-generate-key "John Smith"

(يفضل في الـ **production** إنك تولد المفتاح بـ

gpg --full-generate-key "John Smith") ← ديتيلز أكثر.

- بيطلب اسم ← "John Smith"

- ممكن تسيب password فارغ للتجربة

(لكن في الـ Production لازم كلمة سر قوية)

الناتج:

- مفتاح خاص Private key ← يحتفظ به عندك
- مفتاح عام Public key ← المستخدمون يستخدموه للتحقق

❖ صيغة المفتاح:

• Helm حالياً يفضل الصيغة القديمة (**secring.gpg**) وليس النسخة الجديدة من **GPG v2**

← تحويل المفتاح للصيغة القديمة: **gpg --export-secret-keys > secring.gpg**

4 توقيع Chart

❖ بعد تجهيز المفتاح، نعيد Package مع خيار التوقيع: **helm package nginx --sign --key "John Smith"**

النتيجة:

1. **nginx-0.1.0.tgz** ← Chart مضغوط

```
$ gpg --quick-generate-key "John Smith"
gpg: keybox '/home/vagrant/.gnupg/pubring.kbx' created
About to create a key for:
"John Smith"

Continue? (Y/n) Y
gpg: /home/vagrant/.gnupg/trustdb.gpg: trustdb created
gpg: key 70D5188339885A0B marked as ultimately trusted
gpg: directory '/home/vagrant/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as
'/home/vagrant/.gnupg/openpgp-
revocs.d/20F2395A3176A22DD33DA45470D5188339885A0B.rev'
public and secret key created and signed.

pub  rsa3072 2021-12-01 [SC] [expires: 2023-12-01]
    20F2395A3176A22DD33DA45470D5188339885A0B
uid           John Smith
sub  rsa3072 2021-12-01 [E]

# gpg --full-generate-key "John Smith"

$ gpg --export-secret-keys > ~/.gnupg/secring.gpg
gpg: starting migration from earlier GnuPG versions
gpg: porting secret keys from
'/home/vagrant/.gnupg/secring.gpg' to gpg-agent
gpg: migration succeeded
```

```
$ helm package --sign --key 'John Smith' --keyring ~/.gnupg/secring.gpg ./nginx-chart
Successfully packaged chart and saved it to: /vagrant/nginx-chart-0.1.0.tgz

$ gpg --list-keys

gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model:
PGP
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n,
0m, 0f, 1u
```

← nginx-0.1.0.tgz.prov .2

ملف provenance يحتوي على:

- hash SHA256 للملف
- signature PGP

❖ توضيح الـ Provenance file :

• يحتوي على:

○ اسم Chart

○ النسخة

○ sha256 hash

○ PGP signature من المطور (أنت)

• أي تعديل في الملف ← hash يتغير ← التوقيع يصبح غير صالح

5 رفع المفتاح العام للمستخدمين

❖ لتتمكن من التحقق من signature :

1. المستخدم يحتاج مفاتيح العام Public key

2. يمكن رفعه على keyserver مثل: keyserver.ubuntu.com

• بعد كده، المستخدمون يقدروا يحملوا المفتاح ويستخدموه للتحقق

6 التحقق من التوقيع

❖ بعد تنزيل chart وملف provenance والمفتاح العام: **helm verify nginx-0.1.0.tgz --key mypublickey.asc** النتيجة:

• لو صحيح ← Chart أصلي وآمن

• لو التحقق فشل ← Helm يرفض التثبيت

❖ يمكن دمج التحقق مع التثبيت مباشرة: **helm install nginx-0.1.0.tgz --verify**

```
$ helm verify --keyring ./mypublickey ./nginx-0.1.0.tgz
Signed by: John Smith
Using Key With Fingerprint: 20F2395A3176A22DD33DA45470D5188339885A0B
Chart Hash Verified: sha256:b7d05022a9617ab953a3246bc7ba6a9de9d4286b2e78e3ea7975cc54698c4274

$ gpg --recv-keys --keyserver keyserver.ubuntu.com 8D40FE0CACC3FED4AD1C217180BA57AAFAAD1CA5

$ helm install --verify nginx-chart-0.1.0
```

7 نصائح مهمة

1. دائماً انشر:

○ Chart.tgz

○ Provenance file .tgz.prov

○ مفاتيح العام Public key

2. استخدم كلمة سر قوية على المفتاح الخاص في الإنتاج.

3. Helm يدعم التحقق التلقائي قبل التثبيت ← أمان أعلى.

4. حتى لو الملف .tgz حصل له أي تعديل، التحقق بالكود PGP سيكتشفه فوراً.

✓ الخلاصة

وظيفة

خطوة

تجميع ملفات Chart في ملف واحد .tgz

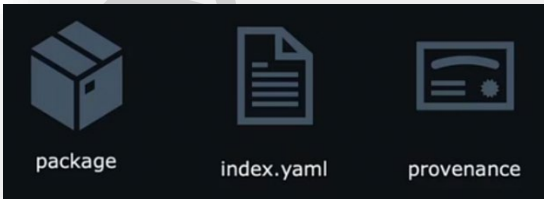
Packaging

- هذه العملية تجعل نشر Helm Charts آمن وموثوق.
- أي تغيير في الملفات أو تزوير ← التحقق يفشل ← لا يتم التثبيت.

Uploading Charts

1 الملفات الأساسية قبل الرفع

❑ بعد ما عملنا Packaging و Signing للـ Chart ، لازم يكون معنا الملفات دي قبل أي رفع:



1. Chart package → nginx-0.1.0.tgz

○ هو الملف المضغوط اللي فيه كل ملفات الـ Chart

2. Provenance file → nginx-0.1.0.tgz.prov

○ فيه signature رقمي للتحقق من أصالة الملف.

3. index.yaml

○ يحتوي على معلومات عن كل الـ Charts الموجودة في Repository

○ فيه URLs لتحديد مكان التحميل .

○ فيه Checksums للـ .tgz files للتحقق من سلامتها.

2 إنشاء مجلد لتجميع الملفات

- أنشئ مجلد جديد لتجميع الملفات التي هترفعها مع بعض، مثلاً:

```
$ ls
nginx-chart  nginx-chart-0.1.0.tgz  nginx-chart-0.1.0.tgz.prov

$ mkdir nginx-chart-files

$ cp nginx-chart-0.1.0.tgz  nginx-chart-0.1.0.tgz.prov nginx-chart-files/

$ helm repo index nginx-chart-files/ --url https://example.com/charts

$ ls nginx-chart-files
index.yaml  nginx-chart-0.1.0.tgz  nginx-chart-0.1.0.tgz.prov
```

3 • توليد index.yaml

- أمر Helm لتوليد index file :

helm repo index nginx-chart-files/ --url https://your-repo-url.com/charts

- شرح التفاصيل:

- nginx-chart-files/ ← المجلد الذي فيه .tgz و .prov
- --url ← عنوان الـ Repository على الإنترنت حيث الملفات ستكون موجودة
- الناتج: ملف index.yaml جوه نفس المجلد.
- محتوى index.yaml :

```
apiVersion: v1
entries:
  nginx-chart:
    - apiVersion: v2
      appVersion: 1.16.0
      created: "2021-12-01T15:29:35.073405539Z"
      description: A Helm chart for Kubernetes
      digest: 2c83c29dc4c56d20c45c3de8eff521fbfb6ef6c0b66854a6f4b5539bebcff879
      maintainers:
        - email: john@example.com
          name: john smith
      name: nginx-chart
      type: application
      urls:
        - https://charts.bitnami.com/bitnami/nginx-chart-0.1.0.tgz
      version: 0.1.0
      generated: "2021-12-01T15:29:35.047718855Z"
```

- URLs : أين يتم تحميل الـ charts

- Checksums : للتحقق من سلامة .tgz

- Entries : كل الـ charts المتاحة في الـ repo

4 • رفع الملفات على Repository

- ❖ يمكنك رفع الملفات لأي خدمة تخزين:

1. خادم ويب خاص ← إذا عندك سيرفر Apache/Nginx

2. Cloud Storage Services

Google Cloud Storage ○

Amazon S3 ○

DigitalOcean Spaces ○

3. GitHub Pages ← رفع الملفات في repo وفتحها كصفحة ثابتة

- خطوات رفع الملفات:

1. ضع كل الملفات (index.yaml, .tgz, .prov) في المكان الذي ستستخدمه كـ Repository

2. احصل على URL للملفات على الإنترنت.

▪ مثال : **https://mybucket.s3.amazonaws.com/nginx-chart-files/**

5 • إضافة Repository للـ Helm المحلي



<https://example-charts.storage.googleapis.com>

❖ بعد مارفعت الملفات على الإنترنت، عايزين نضيفها لقائمة الـ Repos عندنا:

helm repo add my-repo https://mybucket.s3.amazonaws.com/nginx-chart-files/

- my-repo ← الاسم اللي هتسميه للـ repository عندك
- https://... ← URL اللي رفعنا عليه الملفات
- للتأكد: **helm repo list**
- ه يظهر لك الـ repository الجديد في القائمة.

```
$ helm repo add our-cool-charts https://example-charts.storage.googleapis.com

$ helm repo list

NAME                URL
bitnami             https://charts.bitnami.com/bitnami
our-cool-charts     https://example-charts.storage.googleapis.com

$ helm install my-new-release our-cool-charts/nginx-chart
```

6 تثبيت Chart من Repository

- بعد ما أضفنا الـ repo ، نقدر نثبت الـ chart بسهولة: **helm install my-release my-repo/nginx**
- my-release ← الاسم اللي هتعطيه للـ release على الكلاستر
- my-repo/nginx ← اسم repo + اسم الـ chart

ملخص العملية الكاملة

1. جهز الملفات .tgz , .prov
2. أنشئ مجلد جديد وجمعهم فيه
3. استخدم **helm repo index** لتوليد **index.yaml**
4. ارفع الملفات على أي خدمة تخزين على الإنترنت
5. أضف الـ repo محلياً بـ **helm repo add**
6. ثبت Chart باستخدام **helm install**

بعد الخطوة دي، أي مستخدم عنده URL الخاص بالـ repo يقدر يثبت Chart بأمان وبسهولة، ويقدر يتحقق من التوقيع الرقمي باستخدام **provenance file**